

НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ВЫСШИЙ КОЛЛЕДЖ ИНФОРМАТИКИ

Н.Ю.Карпунина, Л.К. Попов

**Изучение языка программирования PYTHON в процессе создания  
компьютерной игры  
(Книга 1)**

НОВОСИБИРСК  
2020

## Оглавление

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Введение .....                                                        | 3  |
| Глава 1. Технология создания компьютерной игры на языке Python .....  | 4  |
| 1.1 Подготовка к работе .....                                         | 4  |
| 1.1.1. Установка Python в ОС Windows .....                            | 4  |
| 1.1.2 Среда IDLE .....                                                | 4  |
| 1.2 Первая программа на языке Python .....                            | 5  |
| 1.3 Целочисленная арифметика .....                                    | 12 |
| 1.4 Графика в Python .....                                            | 13 |
| 1.5 Ветвящийся алгоритм. Условный оператор .....                      | 18 |
| 1.6 Циклический алгоритм .....                                        | 22 |
| 1.6.1 Циклы со счетчиком .....                                        | 23 |
| 1.6.2 Циклы с условием .....                                          | 26 |
| 1.6.3 Неуправляемое движение объекта по экрану .....                  | 28 |
| 1.7 События .....                                                     | 29 |
| 1.8 Функции .....                                                     | 32 |
| 1.8.1 Совмещение неуправляемого и управляемого движений .....         | 36 |
| 1.9 Модули .....                                                      | 38 |
| 1.10 Списки и словари .....                                           | 41 |
| 1.10.1 Словари .....                                                  | 53 |
| 1.11 Сложные рисунки .....                                            | 55 |
| 1.12. Списки в компьютерной игре .....                                | 56 |
| 1.12. Методы и средства работы с текстовой информацией. Строки .....  | 58 |
| 1.13 Текстовые файлы .....                                            | 61 |
| 1.14. Хранение структурированных данных в файлах. Модуль pickle ..... | 66 |
| Глава 2. Виды игровых программ и примеры их реализации .....          | 67 |
| 2.1 Заставка, помощь меню .....                                       | 67 |
| 2.2 Таблица рекордов .....                                            | 71 |
| 2.3 Ввод информации в графическом режиме .....                        | 73 |
| 2.4 Стрельба в компьютерной игре .....                                | 74 |
| 2.5. Лабиринт .....                                                   | 75 |
| Заключение .....                                                      | 78 |
| Литература .....                                                      | 78 |

## Введение

С момента создания ВКИ НГУ студенты-программисты первого курса проходили начальное обучение алгоритмизации и программированию на языке Pascal. В 2020 году было принято решения о переходе на язык Python. Настоящее пособие разработано в связи с необходимостью подготовки учебно-методических материалов для решения этой задачи.

По мнению многих экспертов Python обладает следующими важными свойствами и преимуществами (см. ссылки):

- Python изначально создавался таким образом, чтобы его было легко изучать. Написанные на Python программы читаются, пишутся и понимаются намного проще, чем программы на других языках;
- интерпретатор Python абсолютно бесплатен;
- программное обеспечение Python имеет открытый исходный код. А это означает в частности, что любой пользователь может расширить возможности Python-программ (добавив дополнительную функциональность или упростив решение каких-то задач). Многие пользователи именно так и поступают, в итоге существует большая коллекция доступных для загрузки бесплатных программ;
- Python не только хорошо подходит для изучения программирования, но и применяется тысячами профессионалов по всему миру. Python позволяет реализовать огромное количество задач.

Что касается собственно методики обучения программированию на Python, то в ее основу положена методика начального обучения программированию, разработанная Л.А. Москвиной с соавторами и апробированная и развитая в учебном процессе Высшего колледжа информатики НГУ и получившая в ВКИ рабочее название «Вводный проект». Идея этой методики предельно проста: обучиться программированию можно только в процессе программирования. Поэтому с первых шагов вы будете учиться создавать большую программу, объединенную единым замыслом – компьютерную игру. В этом, собственно, и состоит оригинальность данного подхода. Обычно обучение программированию ведется путем решения небольших, постепенно усложняющихся задач, что отражено в многочисленных учебниках. Мы тоже будем рассматривать по ходу обучения различные примеры. Но все они служат одной цели: обучению самостоятельной разработке законченного программного продукта.

Выбор игры в качестве предметной области программы осознан и неслучаен. Предлагаемая вам методика ориентирована в первую очередь на молодежь – школьников и студентов первых курсов. Игра для них – вид деятельности, в котором они себя ощущают свободно и уверенно. Интерес молодежи к игре неизменен. С другой стороны – игра содержит в себе такое множество условий, правил, алгоритмов, что трудно придумать более благодатное поле для подготовки начинающего программиста.

Курс и соответственно книга состоит из двух частей. Первая часть состоит из занятий, в которых поэтапно изложена технология создания игровой программы на языке программирования Python. Мы начнем обучение «с нуля», не предполагая, что учащийся обладает какими-либо знаниями по информатике и программированию. Задачей первого этапа обучения является всесторонняя подготовка к созданию игровой программы. Вы овладеете основами языка, изучите его графические возможности и основополагающие алгоритмы программирования.

Вторая часть книги содержит материал, который будет полезен при проведении самостоятельных разработок. Здесь вы найдете описание типов игровых программ. Это поможет вам выбрать для самостоятельной разработки игру, соответствующую вашим вкусам, наклонностям и возможностям. Основным методом обучения станет изучение программ, содержащих «заготовки» некоторых игр, полезные алгоритмы и другие необходимые сведения. Но к этому моменту все уже будет зависеть от вас.

Экспериментируйте, пробуйте, ищите новые варианты! Используйте наши советы, и при известной настойчивости вы сможете пригласить своих друзей поиграть в разработанную вами игру.

Книга полезна студентам и школьникам, а также преподавателям информатики и программирования.

## Глава 1. Технология создания компьютерной игры на языке Python

### 1.1 Подготовка к работе

#### 1.1.1. Установка Python в ОС Windows

Для установки Python необходимо скачать установочный файл (например, Python-3.4.1.exe), запустить его на своем компьютере и ответить на все вопросы программы установки, завершая ответ нажатием кнопки «Next».

В первую очередь будет необходимо указать каталог, в который интерпретатор должен быть установлен. По умолчанию он ставится в каталог C:\PythonXX, где XX- номер версии. На этом шаге достаточно нажать кнопку «Next».

Далее программа спросит, нужно ли сохранять резервные копии заменяемых файлов. Если вы ставите Питон впервые, то отметьте вариант «No, do not make backups».

Следующий шаг- выбор устанавливаемых компонентов. Лучше включить установку всего, что предлагается (в том числе, документацию). Полный комплект занимает на жестком диске порядка 20 мегабайт.

Наконец, нужно задать название группы для главного меню. Можно просто нажать «Next» Программа покажет сводную информацию о том, что и куда будет установлено. Еще раз нажмите «Next» и дождитесь окончания установки. Нажмите «Finish» для выхода из программы установки.

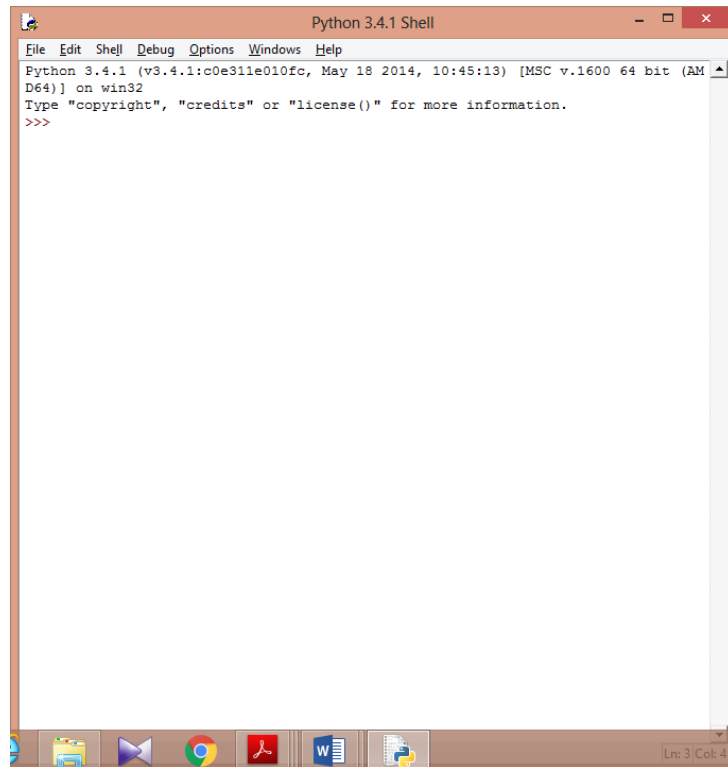
#### 1.1.2 Среда IDLE

Существует несколько способов начать работу с Python. Один из них предполагает применение IDLE, и именно им мы воспользуемся. IDLE (Integrated DeveLopment Environment — интегрированная среда разработки) представляет собой *оболочку* (shell) Python. Оболочка — это средство взаимодействия с программой с помощью ввода текста. И эта самая оболочка позволяет взаимодействовать с Python — именно поэтому в заголовке окна, которое вы увидите после ее запуска, вы обнаружите надпись *Python Shell*.

Значки >>> на картинке — это *приглашение* к вводу команды. Приглашение программа выводит на экран, когда она ждет, чтобы вы что-то ввели. Приглашение >>> говорит нам о том, что Python ожидает от вас дальнейших директив.

Наберем команду print (“Game over”) и после того, как нажмем Enter, обнаружим, что интерпретатор исполнил команду.

```
>>> print (“Game over !”)
Game over
```



В окне Python можно создать и отладить небольшой программный код (как правило фрагмент большой программы), сюда будут выводиться результаты работы программы и сообщения об ошибках в синтаксисе программы. Но для разработки и сохранения программы удобно использовать встроенный в среду IDLE редактор-отладчик. Для этого, после запуска программы в меню следует выбрать команду **File** → **New Window** (Ctrl + N), откроется новое окно. Затем желательно сразу сохранить файл. После того как код будет подготовлен, снова сохраните файл (чтобы обновить сохранение). Ну и наконец, можно запустить программу, выполнив команду меню **Run** → **Run Module** (F5). После этого в исходном окне появится результат выполнения кода. В процессе создания кода здесь можно наблюдать подсветку синтаксиса (в зависимости от значения синтаксической единицы она выделяется определенным цветом). В дальнейшем в нашем изложении мы преимущественно будем использовать среду IDLE.

## 1.2 Первая программа на языке Python

Наша первая программа будет вычислять площадь и периметр квадрата. В программу имеет смысл включать примечания, описывающие, что она делает и как она работает. Это поможет вам (или кому-то другому), глядя на программу спустя некоторое время, понять принцип ее работы.

На компьютерном языке такие примечания называются *комментариями*. Комментарии предназначены только для чтения, к командам для компьютера они отношения не имеют. То есть они являются частью сопроводительной *документации*, и компьютер при выполнении программы их игнорирует. Например,

*(Здесь и далее код программ будет выделен курсивом)*

*# Эта программа вычисляет площадь и периметр квадрата*

Для того, чтобы вычислить площадь и периметр квадрата наша программа должна знать величину стороны квадрата. Информация, которую нужно сообщить программе для ее работы называется данными. Данные нужно занести в память компьютера. Делается это так (первая строка собственно кода программы)

$a=2.5$

Справа в этом выражении расположено значение стороны квадрата. Слева от знака `=` находится идентификатор переменной. Оператор `=` называется оператором присваивания. Все выражение означает значение стороны квадрата (данные) помещается в память компьютера под идентификатор *a*. Идентификатор переменной указывает на место в памяти, где хранится эта информация. С его помощью ее можно найти, использовать для тех или иных целей, наконец, изменить (отсюда наименование – переменная). Идентификатор чаще всего называют именем переменной.

Имена переменных могут быть любыми. Однако есть несколько общих правил их написания:

1. Желательно давать переменным осмысленные имена, говорящие о назначении данных, на которые они ссылаются.
2. Имя переменной не должно совпадать с командами языка (зарезервированными ключевыми словами).
3. Имя переменной должно начинаться с буквы или символа подчеркивания (`_`).
4. Python относится к контекстно чувствительным языкам. То есть, он различает заглавные и строчные буквы *Al* и *al* для него идентификаторы разных переменных.

Ниже, в таблице приводится список ключевых слов, которые нельзя использовать в качестве имен переменных:

|                 |                |               |               |               |
|-----------------|----------------|---------------|---------------|---------------|
| <b>and</b>      | <b>del</b>     | <b>for</b>    | <b>is</b>     | <b>raise</b>  |
| <b>assert</b>   | <b>elif</b>    | <b>from</b>   | <b>lambda</b> | <b>return</b> |
| <b>break</b>    | <b>else</b>    | <b>global</b> | <b>not</b>    | <b>try</b>    |
| <b>class</b>    | <b>except</b>  | <b>if</b>     | <b>or</b>     | <b>while</b>  |
| <b>continue</b> | <b>exec</b>    | <b>import</b> | <b>pass</b>   | <b>yield</b>  |
| <b>def</b>      | <b>finally</b> | <b>in</b>     | <b>print</b>  |               |

Следует обратить внимание на следующее обстоятельство: вид отводимой под переменную памяти зависит от ее типа. Скажем, для вещественного числа нужно отводить память для «знаков после запятой», для целого числа в этом необходимости нет. Особой структуры памяти требует текстовая информация. В «традиционных» языках программирования для этого используется специальная операция: описание переменных. Такой операции в Python нет. Тип переменной определяется операцией присваивания. В нашем примере *a* в дальнейшем будет идентификатором вещественной переменной и отвечать за соответствующую «вещественную» память.

Работая с языком Python, мы точно столкнемся с тремя типами данных:

- целые числа (*integer*) – положительные и отрицательные целые числа, а также 0 (например, 4, 687, -45, 0).
- числа с плавающей точкой (*float point*) – дробные числа (например, 1.45, -3.789654, 0.00453). Примечание: разделителем целой и дробной части служит точка, а не запятая.
- строки (*string*) – набор символов, заключенных в кавычки (например, "ball", "What is your name?", 'dkfjUUv', '6589'). Примечание: кавычки в Python могут быть одинарными или двойными.

На данном этапе мы ограничимся изучением вещественного (float) типа переменных. Особенности работы с другими типами мы рассмотрим позже.

Перейдем к процессу вычислений. Соответствующий код выглядит так:

```
s=a*a  
p=4*a
```

При этом компьютер берет из памяти значение переменной *a*, проводит с ним арифметическую операцию и заносит результат в память под идентификатор *s*, вычисляя при этом значение площади квадрата. Аналогичным образом проводится вычисление его периметра.

Над вещественными числами в языке Python определены четыре арифметических действия

```
+ сложение;  
- вычитание;  
* умножение;  
/ деление;  
** возведение в степень (2**3 это 8)
```

Когда в выражении более одного оператора, последовательность выполнения операций зависит от порядка их следования в выражении, а также от их приоритета. Приоритеты операторов в Питоне полностью совпадают с приоритетами математических операций.

Самый высокий приоритет у скобок, которые позволяют изменять последовательность выполнения операций. Таким образом, операции в скобках выполняются в первую очередь. Например,  $2 * (3-1)$  равно 4,  $(1+1) ** (5-2) = 8$ . Скобки удобно использовать и для того, чтобы выражения было легче читать, даже если их наличие в выражении никак не отражается на результате:  $(100*20)/80$ .

Следующий приоритет у операции возведения в степень, поэтому  $2**1+1$  равно 3, а не 4, и выражение  $3*1**3$  даст результат 3, а не 27.

Умножение и деление имеют одинаковый приоритет, более высокий, чем у операций сложения и вычитания.  $2*3-1$  равно 5, а не 4.

Операторы с одинаковым приоритетом выполняются слева направо. Так что в выражении  $100*20/80$  умножение выполняется первым (выражение приобретает вид  $2000/80$ ); затем выполняется деление, выдающее в результате значение 25. Если бы операции выполнялись справа налево, то результат получился бы другим.

Теперь в нашей программе все как будто готово: площадь и периметр вычислены. Осталось вывести полученные результаты на экран. Для этого используют оператор `print()`:

```
print('Площадь квадрата', s)  
print('Периметр квадрата', p)
```

В круглых скобках список выводимой информации. До запятой в кавычках – текст, на языке программирования – строка, строковая переменная (о них мы поговорим позже). После запятой – идентификатор переменной, значение которой мы собираемся вывести на экран. В итоге наша программа приобретает следующий вид:

```
# Эта программа вычисляет площадь и периметр квадрата  
a=2.5  
s=a*a
```

```
p=4*a
print('Площадь квадрата',s)
print('Периметр квадрата',p)
```

Запустим программу, выполнив команду меню **Run** → **Run Module** (F5). При этом мы получим приглашение сохранить файл. Сохраним его под именем *kvadrat*. Расширение *py* Python присвоит ему автоматически. В итоге в главном окне Python Shell появятся следующие результаты:

```
Площадь квадрата 6.25
Периметр квадрата 10.0
```

Как будто все работает. Но данная программа может использоваться только самим ее автором, да и то только в учебных целях. Для того, чтобы ею мог воспользоваться другой человек (пользователь), который имеет право ничего не знать о ее коде, программу следует доработать. В первую очередь надо предоставить пользователю увидеть результаты работы программы. Сейчас, если пользователь не будет пользоваться редактором Python (это дело программиста), а запустит программу щелчком мыши по имени файла в Проводнике Windows, перед ним на мгновение мелькнет окно *py.exe*, и он ничего не увидит. Необходимо остановить исполнение программы. Это можно сделать, разместив в конце программы оператор *input()*. Этот оператор останавливает исполнение программы до нажатия клавиши *Enter*, и пользователь может спокойно рассмотреть результаты работы программы:

```
# Эта программа вычисляет площадь и периметр квадрата
a=2.5
s=a*a
p=4*a
print('Площадь квадрата',s)
print('Периметр квадрата',p)
input()
```

Далее: пользователь после запуска программы должен увидеть ее предназначение. Комментарии (первая строка) на экране не отображается. Выполним эту задачу с помощью оператора *print()*.

```
# Эта программа вычисляет площадь и периметр квадрата
print("Эта программа вычисляет площадь и периметр квадрата")
a=2.5
s=a*a
p=4*a
print('Площадь квадрата',s)
print('Периметр квадрата',p)
input()
```

Предоставим теперь пользователю возможность самому ввести данные (сторону квадрата) с клавиатуры. Ввод данных с клавиатуры в программу (начиная с версии Python 3.0) осуществляется с помощью функции *input()*. Когда данная функция выполняется, то выполнение программы останавливается в ожидании данных, которые пользователь должен ввести с помощью клавиатуры. После ввода данных и нажатия *Enter*, функция *input()* завершает свое выполнение и возвращает результат, который представляет собой строку символов, введенных пользователем.



```
>>> input()
1234
'1234'
```

Когда выполняющаяся программа предлагает пользователю что-либо ввести, то пользователь может не понять, чего от него хотят. Надо как-то сообщить, ввод каких данных ожидает программа. С этой целью функция `input()` может принимать необязательный аргумент-приглашение строкового типа; при выполнении функции сообщение будет появляться на экране и информировать человека о запрашиваемых данных.

```
>>> input("Введите номер карты: ")
Введите номер карты: 98765
'98765'
>>> input('Input your name: ')
Input your name: Sasha
'Sasha'
>>>
```

Из примеров видно, что данные возвращаются в виде строки, даже если было введено число. Начиная с версии Python 3.0, если требуется получить число, то результат выполнения функции `input()` изменяют с помощью функций `int()` или `float()`.

```
>>> input('Введите число: ')
Введите число: 10
'10'
>>> int(input('Введите число: '))
Введите число: 10
10
>>>
```

Результат, возвращаемый функцией `input()`, обычно присваивают переменной для дальнейшего использования в программе. В нашем случае уместно использовать именно этот прием

```
a=float(input("Введите сторону квадрата\n"))
```

Символы `\n` означают перевод строки. Ввод числа будет проводиться на следующей за приглашением «Введите сторону квадрата» строке.

В итоге получается следующая программа:

```
# Эта программа вычисляет площадь и периметр квадрата
print("Эта программа вычисляет площадь и периметр квадрата")
a=float(input("Введите сторону квадрата\n"))
s=a*a
p=4*a
print('Площадь квадрата',s)
print('Периметр квадрата',p)
input()
```

Предположим, теперь, что нам нужно вычислить диагональ квадрата. Для этого необходима операция извлечения квадратного корня (`sqrt`). Математические функции и

константы ( $\pi$ ,  $e$ ) хранятся в библиотеке `math`. Информацию о ней можно получить в справке Python. Импорт функций из библиотеки производится следующим образом:

```
from math import sqrt
```

Программа будет иметь следующий вид (вывод значений периметра и диагонали реализован с помощью одного оператора `print()`). Мы будем пользоваться этой возможностью и в дальнейшем).

```
# Эта программа вычисляет площадь и периметр квадрата  
from math import sqrt  
print('Эта программа вычисляет площадь и периметр квадрата')  
a=float(input("Введите сторону квадрата\n"))  
s=a*a  
p=4*a  
d=sqrt(2)*a  
print('Площадь квадрата',s)  
print('Периметр квадрата',p,', Диагональ квадрата',d)  
input()
```

До сих пор для вставки переменных в строки мы поступали так:

```
name = 'Леонид'  
print ('Меня зовут', name, 'и я написал эту книгу.')
```

Запустив этот код, получим строку:

*Меня зовут Леонид и я написал эту книгу.*

Но существует и другой способ вставки переменных в строки, позволяющий лучше контролировать вид конечного результата, особенно чисел. Это *форматирующие строки* (format strings), содержащие символ процента (%). Предположим, нам нужно вставить строковую переменную в середину вывода, реализуемого инструкцией `print`, как это было сделано в предыдущем примере. С форматирующей строкой это будет выглядеть следующим образом:

```
name = 'Леонид'  
print ('Меня зовут %s и я написал эту книгу' % name)
```

Символ `%` встречается в двух местах. Прежде всего в середине строки, куда мы хотим вставить переменную, и затем после строки, сообщая интерпретатору Python, что следом идет предназначенная для вставки переменная. Запись `%s` означает вставку строковой переменной. Если нам нужно вставить целое число, мы напишем `%i`; для вставки десятичного числа используется запись `%f`.

Вот еще пара примеров:

```
age = 70  
print ('Мне %i лет.' % age)
```

Вот что получится после запуска этого кода:

*Мне 70 лет.*

Другой пример:

```
average = 75.6
```

```
print ('Среднее математического теста составляет %f процентов.' % average)
```

Запустив код, получим:

*Среднее математического теста составляет 75.600000 процентов.*

Именно сочетания символов %s, %f и %i называются *форматирующими строками* и определяют вид выводимой переменной.

При выводе чисел хотелось бы управлять их видом. Для этого нужно знать:

- количество знаков после запятой;
- вид записи (обычная или экспоненциальная);
- наличие ведущих или конечных нулей;
- наличие знака + или −.

Форматирующие строки позволяют делать как это, так и многое другое.

Рассмотрим пример. Скажем, нужно вывести десятичное число с двумя знаками после десятичной точки. Запустите следующий код в интерактивном режиме:

```
>>> dec_number = 12.3456
>>> print ('Сегодня %.2f градуса.' % dec_number)
Сегодня 12.35 градуса
```

В середине инструкции print находится форматирующая строка. Но на этот раз мы записали не %f, а %.2f. Это заставило интерпретатор Python вывести два знака после десятичной точки в числе с плавающей точкой. (Обратите внимание, что интерпретатор Python округлил число до двух цифр после точки, а не просто отбросил лишние разряды.) Второй знак %, стоящий после строки, сообщает интерпретатору Python, что следом идет информация, которую нужно показать. Вид выводимого числа задается форматирующей строкой.

Для вывода целых чисел используйте форматирующую строку %d или %i по своему выбору.

```
>>> number = 12.67
>>> print '%i' % number
12
```

Обратите внимание, что на этот раз число не округляется. От него просто *отбрасывается дробная часть*. В случае округления результат был бы равен 13, а не 12. При форматировании целых чисел у них отбрасывается дробная часть, в то время как числа с плавающей точкой при форматировании округляются.

Форматирующая строка для вещественных чисел включает в себя букву f в верхнем (%F) или нижнем (%f) регистре:

```
>>> number = 12.3456
>>> print ('%f' % number)
```

12.345600

Запись %f округляет число до шести цифр после десятичной точки. Добавив перед f .n, где n –любое целое число, вы оставите n знаков после точки:

```
>>> print ('%.2F' % number)
12.35
```

Как видите, число 12.3456 было округлено до двух знаков после точки: 12.35.

Чтобы закрепить полученные знания, выполните самостоятельно следующие задания. Во всех заданиях данные должны вводиться с клавиатуры, а результаты выводиться на экран.

### Задания для самостоятельной работы.

1. Напишите программу для вычисления площади круга и периметра окружности. Число  $\pi$  обозначается как  $pi$ . Его необходимо импортировать из библиотеки *math*. ( $S=Pi*R**2$   $L=2*Pi*R$ )
2. Вычислить площадь и периметр прямоугольного треугольника, если известны длины его катетов. ( $S=a*b$   $P=a+b+sqrt(a**2+b**2)$ )
3. Тело движется равноускоренно. Ускорение и начальная скорость известны (вводятся с клавиатуры). Какой путь пройдет тело за время  $t$ ? ( $S = V_0 + ((a*t^2) / 2)$  )
4. Вычислить период колебаний маятника длины  $L$ . ( $t = 2*Pi*sqrt(L/g)$  ) $g=9.8$
5. Даны стороны треугольника. Вычислить его площадь.
6. Определить силу притяжения между двумя телами с разной массой, находящимися на расстоянии  $R$  друг от друга. ( $F = GMm/D^2$ )  
 $G=6.67*10^{-11}$

### 1.3 Целочисленная арифметика

Этот раздел готовит нас к работе с компьютерной графикой. Единицей измерений на экране является экранная точка – пиксель. Она неделима – не бывает половины или трети точки. Поэтому для работы с графикой используются целые числа. Арифметические операции, проводимые над целыми числами, обладают некоторыми особенностями. В результате арифметического действия над целым числом должно также получиться целое число. Этому требованию безусловно удовлетворяют операции сложения, умножения и вычитания. Сложнее с делением. Результатом операции  $5/2$  будет вещественное число 2.5. Поэтому для целых чисел предусмотрена операция «// - деление без остатка» , в результате операции  $5//2$  Python вернет число 2, просто отбросив дробную часть числа. В результате операции  $5\%2$  вы получите остаток от деления 1.

Следующий пример поможет вам освоить целочисленную арифметику. Предположим вас есть некоторое количество денег в рублях. Вам известна цена бутылки пепси-колы тоже в рублях. Программа должна определить, сколько бутылок вы можете купить и какую сдачу вы получите.

```
#Эта программа поможет контролировать ваши расходы
print('Программа вычисляет количество приобретенного товара и сдачу')
dengi=int(input('Сколько у вас денег?\n'))
zena=int(input('Введите цену товара\n'))
kolvo=dengi//zena
```

```
dengi=dengi%zena
print('Вы купили',kolvo,'бутылок пепси. Ваша сдача составит',dengi,'рублей')
input()
```

Возможно вы заметили, что мы не предусмотрели переменной для сдачи. Мы используем для вычисления сдачи ту же переменную *dengi*. По этой команде Python выполняет следующие действия: берет из памяти переменную *dengi*, вычисляет остаток от деления ее на переменную *zena* и результат записывают в то же место в памяти, где хранилось значение переменной *dengi*. Старое значение при этом стирается. Такой способ использования оператора присваивания очень распространен в программировании (его также называют переприсваиванием), и мы часто будем им пользоваться.

Следующая задача формулируется так:

Дан интервал времени в секундах. Определить, сколько это минут и секунд.

Вот ее решение:

```
#Программа по количеству секунд определяет сколько это минут и секунд
print('По количеству секунд найти количество минут и секунд')
sec=int(input('Введите количество секунд\n'))
minut=sec//60
sec=sec%60
print('Это',minut,'минут и',sec,'секунд')
input()
```

Мы надеемся, что вы поняли правила работы с целыми числами. Для закрепления своих знаний выполните следующие упражнения:

### Задания для самостоятельной работы

1. Дан интервал времени в секундах. Определить, сколько это часов, минут и секунд.
2. Дано текущее время в часах, минутах и секундах и интервал времени – также в часах, минутах и секундах. Что будут показывать часы по прошествии этого интервала времени. Например: сейчас 22 часа 35 минут 56 секунд. Сколько времени будет через 3 часа 40 минут и 15 секунд. Учтите, что могут наступить следующие сутки.
3. Задача почтальона. Вы знаете номер квартиры, знаете, что в доме 9 этажей и на каждом этаже 4 квартиры. В какой подъезд вам следует войти и какую кнопку лифта нажать, чтобы попасть в эту квартиру.
4. Вводится целое число. Вывести число, обратное по порядку составляющих его цифр введенному. Например, введено 3425, надо вывести 5243.

## 1.4 Графика в Python

Разработка компьютерной игры ведется с помощью графических средств. Создавать на своем компьютере графику (и добавлять к ней звук) не так-то просто. Вам придется иметь дело с операционной системой, видеокартой и большим количеством низкоуровневого кода, о котором нам пока не хочется даже говорить. Поэтому для простоты решения мы воспользуемся Python-модулем, который называется Pygame.

Первым делом нам нужно окно, в котором будет нарисована вся графика. Вот очень простая программа, которая нам его создаст. Принцип ее работы ясен из комментариев.

```
import pygame #Импортируем модуль pygame
```

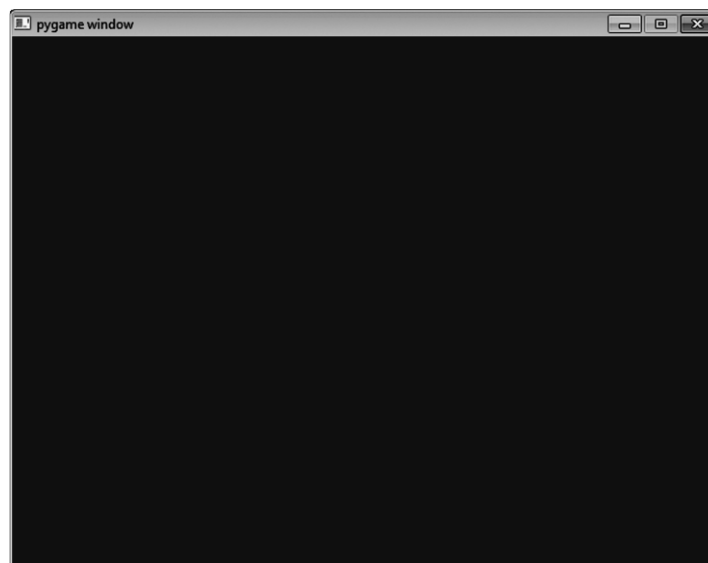
```

pygame.init()#Инициализируем модуль pygame
screen = pygame.display.set_mode([640, 480])#задаем размеры графического окна
running = True
while running:
    for event in pygame.event.get():#просматриваем очередь событий
        if event.type == pygame.QUIT:#если закрыто окно
            running = False #Выходим из цикла
pygame.quit() #закрываем графическое окно

```

Код, следующий за оператором while пока не совсем понятен. Мы вернемся к его обсуждению при изучении циклического алгоритма. Обращаем внимание на отступы в этом коде. Их требует синтаксис Python. Подробнее их необходимость также будет рассмотрена ниже.

В результате мы получим графическое (пока еще пустое) окно.



Перейдем к созданию простейших рисунков (так называемых, графических примитивов). Следует иметь ввиду следующие правила:

- Положение объектов на графическом окне задается в пикселях.
- Началом координат является верхний левый угол окна.
- Чем больше значение координаты X, тем **левее** положение точки;
- Чем больше значение координаты Y, тем **ниже** положение точки.

Следующая программа рисует прямоугольник. В ее тексте приведены необходимые комментарии.

```

import pygame
pygame.init()
screen = pygame.display.set_mode([640,480])
screen.fill([255, 255, 255])#белый экран
#Размеры и положение прямоугольника
width = 200
height = 100
top = 100
left = 100
# рисуем прямоугольник
pygame.draw.rect(screen, [255,0,0], [left, top, width, height], 0)

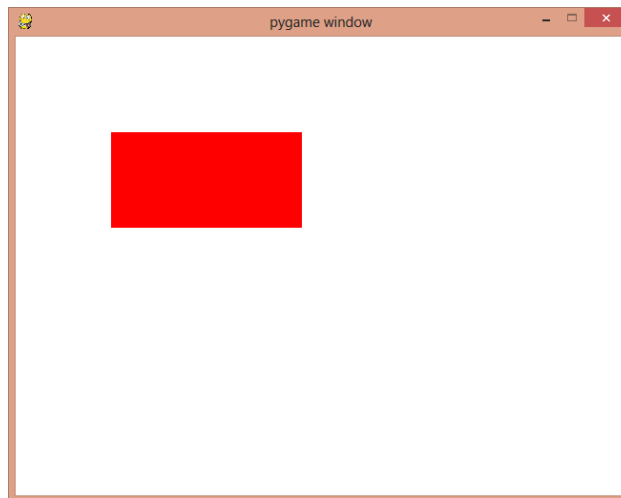
```

```

#screen - на чем рисуем
#[255,0,0] - цвет RGB, red
#[left, top, width, height]- координаты левого верхнего угла, длина ширина
# 0 - сплошная заливка, 1,2 и т.д - ширина линии контура, заливка отсутствует
pygame.display.flip()# Вывод изображения в графическое окно
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()

```

На рисунке вы видите результат работы этой программы.

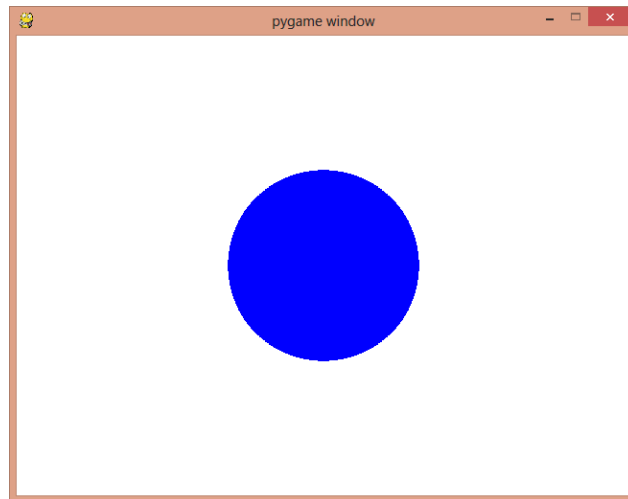


Следующая программа рисует круг.

```

import pygame
pygame.init()
from pygame.color import THECOLORS # Эта библиотека позволяет задавать цвета
screen = pygame.display.set_mode([640,480])
screen.fill([255,255,255]) # Белый экран
pygame.draw.circle(screen, THECOLORS['blue'],[320,240], 100, 0)
#screen - на чем рисуем
#THECOLORS['red'] - цвет
#[320,240] - координаты центра круга
#100 - радиус
# 0 - сплошная заливка, 1,2 и т.д - ширина линии контура, заливка отсутствует
pygame.display.flip() # Эта команда выводит изображения в окно.
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()

```

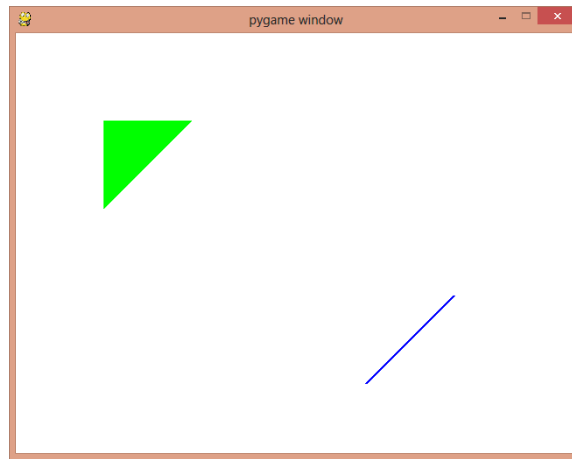


Ниже приведен код программы, рисующей линию и треугольник.

```
import pygame
pygame.init()
from pygame.color import THECOLORS
triangle=[[100,100],[100,200],[200,100]]#координаты вершин треугольника
screen = pygame.display.set_mode([640,480])
screen.fill([255,255,255]) # fill screen with white
pygame.draw.polygon(screen,[0,255,0],triangle,0)#рисует треугольник
#[0,255,0] - цвет
#triangle - координаты вершин
#0 - заполнение
pygame.draw.lines(screen, THECOLORS['blue'],False,[[500,300],[400,400]],3)
#рисует линию
#THECOLORS[] - цвет
#FALSE - линия незамкнута
#[[500,300],[400,400]] - координаты концов линии
#3 - толщина линии
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```

Результат работы программы:



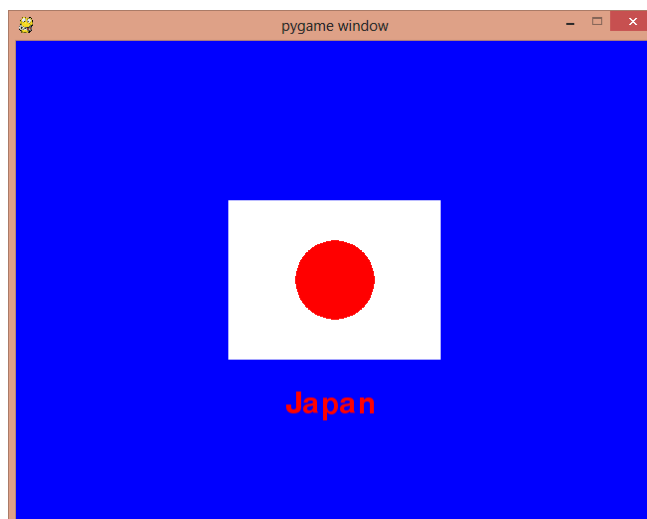


Используем теперь накопленные знания для создания конкретного рисунка. Это будет японский флаг на фоне голубого неба. Под рисунком должна быть подпись – «Япония».

Предварительное замечание о порядке рисования. Если маленький объект находится на фоне большого, то сначала рисуют большой, а потом маленький. Например, сначала небо, потом белое полотнище, потом восходящее солнце. А не наоборот! Рассмотрим код этой программы.

```
import pygame, sys
pygame.init()
from pygame.color import THECOLORS
from pygame import font#импорт шрифтов
screen = pygame.display.set_mode([640,480])
screen.fill([0,0,255])
pygame.draw.rect(screen, [255,255,255], [640//3, 480//3, 640//3, 480//3], 0)
pygame.draw.circle(screen, THECOLORS["red"],[320,240], 40, 0)
font = pygame.font.Font(None, 45)#задаем шрифт
#None - шрифт по умолчанию
#45 - размер символа
text = font.render("Japan",1, THECOLORS ["red"])
#создаем поверхность text
#"Japan" - надпись
#1- сглаживание 1 -есть, 0 -нет
#THECOLORS ["red"] - цвет надписи
screen.blit(text, [270, 350])
#копирование с поверхности text на поверхность screen
#text - копируемая поверхность
#[270, 350] - координаты левого верхнего угла надписи
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```

Результат работы программы:



Если в реальной жизни вас попросят нарисовать картину, первым делом вы спросите: «На чем ее рисовать?» В модуле Pygame *поверхность* (surface) – это то, на чем вы рисуете. *Поверхность отображения* (display surface) – то, что вы видите на экране. Именно она в наших программах получила имя `screen`. Но в Pygame-программе может быть множество поверхностей, причем существует возможность копировать изображения с одной поверхности на другую. Более того, сами поверхности можно поворачивать и масштабировать (делать больше или меньше).

В программе «Японский флаг» таких поверхностей две. Оператор `text = font.render("Japan", 1, THECOLORS ["red"])` создает поверхность для отображения текста. Затем оператор `screen.blit(text, [270, 350])` копирует поверхность `text` на поверхность `screen`. Результат выводится на экран командой `pygame.display.flip()`.

Существуют две копии поверхности отображения. Программисты говорят, что поверхность отображения имеет *двойную буферизацию* (double-buffered). Именно благодаря этому на экране не появляются наполовину нарисованные фигуры и изображения. Круги, инопланетяне и другие объекты рисуются в буфере, на который затем «переходит» поверхность отображения, образуя полностью готовые рисунки.

### Задания для самостоятельной работы

1. Нарисуйте российский флаг.
2. Нарисуйте флаг Франции.
3. Нарисуйте флаг Чехии.
4. Нарисуйте олимпийский флаг.
5. Нарисуйте зимний пейзаж (голубое небо, белый снег, домик, снеговика, елку из треугольников, солнце и все, что придет в голову на эту тему).

### 1.5 Ветвящийся алгоритм. Условный оператор

Напомним, что конечной нашей целью является создание компьютерной игры. Игра протекает по правилам. А правила – это свод различных условий. И в игре, и в жизни, и в программировании наши действия чаще всего зависят от входных данных. Поясним это на простом примере. Предположим, вы проснулись утром и чувствуете, что у вас болят голова и горло. Вы измеряете свою температуру и в случае, если она выше  $37^{\circ}\text{C}$ , вызываете врача и ложитесь в постель, а если не выше – делать нечего, идете в школу. Такой, зависящий от начальных условий алгоритм называется ветвящимся алгоритмом. Для его реализации в

языке Python предусмотрен условный оператор *if* со следующим синтаксисом (правилами написания):

```
if <условие1>:
    <оператор1>
elif <условие2>:
    <оператор2>
else:
    <оператор3>
```

Python действует следующим образом: он проверяет первое условие. Если оно выполнено, он выполняет действия, обозначенные как <оператор1>, и дальнейшие проверки не проводятся. Обращаем внимание на отступ перед оператором. В Python он выполняет роль операторных скобок (в языке Pascal эту роль выполняют ключевые слова *begin* *end*, в языке C – фигурные скобки).

Если надо проверить еще какое-нибудь условие, эту проверку выполняет инструкция *elif*. Если условие 2 выполнено, выполняется <оператор2>. Количество инструкций *elif* зависит от количества необходимых проверок. Наконец, если ни одно из условий не выполнено, используется инструкция *else* и выполняется <оператор3>. Инструкции *elif* и *else* не являются обязательными.

Условия (логические выражения) формулируют с помощью операторов сравнения. Они приведены в следующей таблице:

| Оператор | Значение         | Пример | Истинность |
|----------|------------------|--------|------------|
| ==       | Равно            | 5==5   | True       |
| !=       | Не равно         | 8!=5   | True       |
| >        | Больше           | 3>10   | False      |
| <        | Меньше           | 5<8    | True       |
| >=       | Больше или равно | 5>=19  | False      |
| <=       | Меньше или равно | 5<=5   | True       |

Логические выражения возвращают только два значения **True** или **False**, которые ведут себя как целые числа 1 и 0 соответственно. Значение логического выражения в операторе условия может быть инвертировано с помощью оператора **not**. Несколько логических условий перехода можно объединить в одно с помощью операторов **and** и **or**.

Поясним действия условного оператора на примерах. Начнем с действий, которые последуют за измерениями температуры тела. Код этой программы будет выглядеть следующим образом:

```
# Действия студента после измерения температуры тела
print('Измерьте и введите температуру тела')
t=float(input('Введите температуру\n'))
if (t>=37):
    print('Вызывайте врача')
else:
    print('Идите в колледж')
input()
```

Приведем еще один пример. Решим следующую задачу:

Дан круг радиуса  $R$  с центром в начале координат. Определить, принадлежит ли точка, координаты которой вводятся с клавиатуры, данному кругу.

Как мы знаем из геометрии, условием принадлежности точки кругу является соотношение  $R^2 \leq x^2 + y^2$ , где  $x, y$  – координаты точки. Следовательно, программа, использующая это условие, должна выглядеть так:

```
print("Попадает ли точка в круг")
x=float(input("Введите x\n"))
y=float(input("Введите y\n"))
r=float(input("Введите r\n"))
if (x*x+y*y)<=r*r: print("Yes")
else: print("No")
input()
#Если выполняется один оператор,
#он может находиться в той же строке, что и условие
```

Усложним теперь задачу. Выясним, принадлежит ли точка верхней половине этого круга (рисунок). В этом случае необходимо одновременное выполнение двух условий:  $R^2 \leq x^2 + y^2$  и  $y \geq 0$ . В этом случае в программировании используют операцию *and* – логическое «и». Программа будет выглядеть так:

```
print("Попадает ли точка в круг")
x=float(input("Введите x\n"))
y=float(input("Введите y\n"))
r=float(input("Введите r\n"))
if ((x*x+y*y)<=r*r) and (y>=0): print("Yes")
else: print("No")
input()
```

Введем еще одно усложнение. Попробуем выяснить, попадает ли точка в верхний правый или в нижний левый квадранты круга. В этом случае должно быть выполнено одно из двух условий:  $(r*r) \leq (x*x + y*y)$  и  $((y>=0) \text{ and } (x>=0))$  или  $((y<=0) \text{ and } (x<=0))$ . Теперь нам понадобится операция *or* – логическое или. В результате получим:

```
print("Попадает ли точка в круг")
x=float(input("Введите x\n"))
y=float(input("Введите y\n"))
r=float(input("Введите r\n"))
if ((x*x+y*y)<=r*r) and (((y>=0) and (x>=0)) or ((y<=0) and (x<=0))):
    print("Yes")
else: print("No")
input()
```

Следующая программа находит корни квадратного уравнения  $ax^2+bx+c=0$ . Проанализируйте ее самостоятельно.

```
from math import sqrt
a=float(input("a= \n"))
b=float(input("b= \n"))
c=float(input("c= \n"))
```

```

disc=b*b-4*a*c
# Вычисление дискриминанта
if (a==0) and (c==0) and (b==0):
    print("Бесконечное множество решений")
elif (a==0) and (b==0):
    print("Нет решений");
elif (a==0):
    x1=-c/b
    print("Один корень x1= ", "%.4f" %(x1))
elif (disc<0):
    print("Нет решений")
elif (disc==0):
    x1=-b/(2*a)
    print("Один корень x1= ", "%.4f" %(x1))
else:
    x1=(-b+sqrt(disc))/(2*a)
    x2=(-b-sqrt(disc))/(2*a)
    print("Первый корень x1= ", "%.4f" %(x1))
    print("Второй корень x2= ", "%.4f" %(x2))
input()

```

Для приобретения навыков работы с условным оператором полезны упражнения с применением графики. Например: графическое отображение задачи о попадании точки в верхний правый или в нижний левый квадранты круга.

```

#Попадает ли точка в круг
import pygame, sys, random
pygame.init()
from pygame.color import THECOLORS
from pygame import font
screen = pygame.display.set_mode([640,480])
font = pygame.font.Font(None, 45)
running = True
r=200
while running:
    screen.fill(THECOLORS['white'])
    pygame.draw.lines(screen, THECOLORS['blue'], False, [[320,0],[320,480]], 2)
    pygame.draw.lines(screen, THECOLORS['blue'], False, [[0,240],[640,240]], 2)
    pygame.draw.circle(screen, THECOLORS['blue'], [320,240], r, 2)
    #Рисуем оси координат и окружность
    x=random.randint(0,640)
    y=random.randint(0, 480)
    #Задаем координаты точки
    pygame.draw.circle(screen, THECOLORS['red'], [x,y], 2, 0)
    if ((r*r)>((x-320)*(x-320)+(y-240)*(y-240))) and (((y<240) and (x>320)) or ((y>240)
and (x<320))):
        text = font.render("Попадает", 1, THECOLORS ["blue"])
    else:
        text = font.render("Не попадает", 1, THECOLORS ["magenta"])
    screen.blit(text, [370, 400])
    pygame.time.delay(2000)
    for event in pygame.event.get():

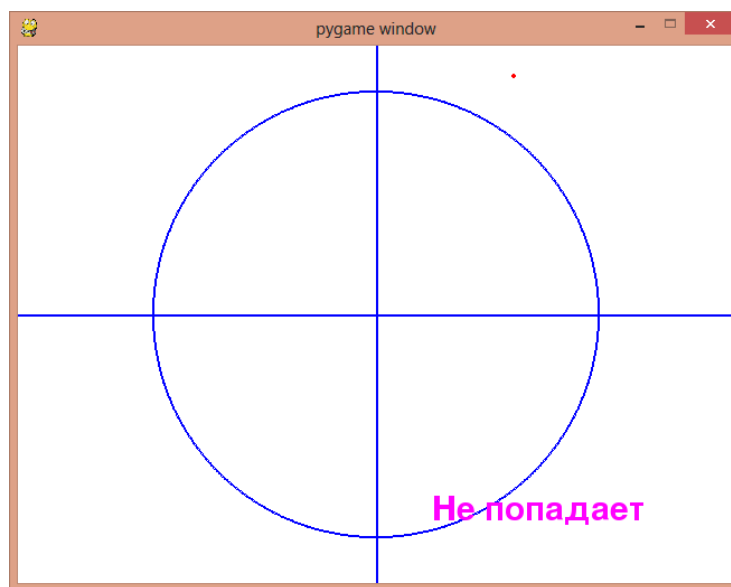
```

```

if event.type == pygame.QUIT:
    running = False
pygame.display.flip()
pygame.quit()

```

В этой программе используется датчик случайных чисел. С этой целью импортируем модуль *random*. Оператор `x=random.randint(0,640)` случайным образом присвоит *x* значения в диапазоне от 0 до 640.



### Задания для самостоятельной работы

1. Кирпич имеет форму параллелепипеда со сторонами *a, b, c*. Определить, пройдет ли он в отверстие радиуса *R*.
2. Мария Васильевна – пенсионерка, и пользуется льготами на оплату электроэнергии. За первые 100 киловатт-часов она платит по 10 копеек за киловатт-час, за вторую сотню – по 20 копеек и за все киловатт-часы свыше 200 – по 30 копеек. Написать программу, которая после ввода с клавиатуры количества израсходованных киловатт-часов сообщала бы Марии Васильевне сумму ее долга за электроэнергию.
3. Даны (вводятся с клавиатуры) три стороны треугольника. Определить тип треугольника. Проверить возможные ошибки ввода.

### Графические упражнения

4. Попадает круг вверх, вниз и на границу.
5. Два круга: попадает ли один в другой
6. Два прямоугольника: попадает ли один в другой

## 1.6 Циклический алгоритм

В жизни нам часто приходится совершать повторяющиеся действия. В компьютерных играх это тоже необходимый элемент. Любое движение в такой игре – это повторяющееся перемещение на определенное расстояние. Ходьба с точки зрения программирования – это циклическая перестановка ног. Естественно, что среди алгоритмов программирования есть циклические алгоритмы. Циклический алгоритм или цикл – это последовательность действий, которая повторяется более одного раза.

Циклы бывают двух видов. Иногда мы заранее знаем, сколько раз нам нужно совершить то или иное действие. Например, пройти сто шагов и остановиться. Такой цикл называют циклом со счетчиком. А иногда нам нужно прекратить действие или, как говорят программисты, выйти из цикла при выполнении некоторого условия. Например, дойти до автобусной остановки. Условием прекращения ходьбы является совпадение координат остановки с вашими координатами. Такой цикл называют циклом с условием.

#### 1.6.1 Циклы со счетчиком

Для реализации данных алгоритмов в языке Python предусмотрены необходимые операторы. Начнем с оператора цикла со счетчиком. Такие циклы называют также циклами `for`, так как во многих языках, в том числе в Python, для создания таких циклов применяется ключевое слово `for`.

Попробуем применить такой цикл на практике. Рассмотрим следующий пример:

```
#Простейший цикл for
for i in [1, 2, 3, 4, 5]:
    print("hello")
```

Результатом работы этой программы будет следующая распечатка:

```
hello
hello
hello
hello
hello
```

Программа вывела слово «hello» пять раз, хотя инструкция `print` у нас всего одна. Почему же так получилось? Первая строка (`for i in [1, 2, 3, 4, 5]:`) в переводе означает следующее:

- 1) переменная `i` сначала имеет значение 1 (то есть `i = 1`);
- 2) цикл выполняет расположенный ниже блок команд один раз для каждого значения из списка (Списком в данном случае являются цифры в квадратных скобках.);
- 3) на каждом следующем шаге цикла переменной `i` присваивается следующее значение из списка.

Вторая строка (`print("hello")`) представляет собой блок, который интерпретатор Python выполняет на каждом проходе цикла. Циклу `for` блок кода необходим, чтобы программа знала, какие действия ей нужно выполнять на каждом проходе. Этот блок (часть кода с отступом) называется *телом цикла*.

Следующий пример: выведем на экран таблицу умножения. Для этого внесем в нашу программу небольшие изменения:

```
for i in [1, 2, 3, 4, 5]:
    print(i, "*8=", i*8)
```

Получим следующий результат:

```
1 *8= 8
2 *8= 16
3 *8= 24
4 *8= 32
5 *8= 40
```

В предыдущем примере цикл повторялся всего 5 раз. А что делать, если цикл нужно запустить 100 или 1000 раз? Для этих целей используют функцию *range()*. Она позволяет указывать только начальное и конечное значения, создавая за вас все остальные значения диапазона. Выведем таблицу умножения при помощи функции *range()*.

```
for i in range(1,5):  
    print (i,"*8=",i*8)
```

Результат работы программы будет прежним, за исключением последней строки. Она отсутствует. Дело в том, что функция *range (1, 5)* дает нам список [1, 2, 3, 4]. Это нужно иметь в виду при практическом программировании.

В функции *range()* вовсе не обязательно указывать два числа, как мы это делали в предыдущих примерах. Можно обойтись одним: *for i in range(5):*. Это аналогично записи: *for i in range(0, 5)*. В обоих случаях получается список цифр: [0, 1, 2, 3, 4]. На самом деле многие программисты начинают циклы с 0, а не с 1. Написав *range(5)*, вы получите 5 итераций цикла, что очень легко запомнить. Просто следует знать, что первое значение *i* равно 0, а не 1, а последнее 4, а не 5.

До этого момента на каждой итерации мы прибавляли к переменной цикла единицу. А что делать, если мы хотим прибавлять на каждом шаге 2? Или 5, или даже 10? А как насчет обратного отсчета? В функцию *range()* можно добавить еще один *аргумент*, позволяющий менять величину шага, который по умолчанию равен 1.

```
for i in range(1, 10, 2):  
    print (i)
```

Результат

1  
3  
5  
7  
9

Если третий аргумент сделать отрицательным, мы можем реализовать обратный отсчет или, как иногда говорят, нисходящий цикл.

```
for i in range(10, 1, -1):  
    print (i)
```

Во всех предыдущих примерах в качестве переменной цикла фигурировало число. В терминах программирования цикл *осуществлял перебор* списка чисел. Но список может состоять из чего угодно. Можно взять список символов (строку). А можно воспользоваться списком строк или еще чем-нибудь.

Как это работает, лучше всего посмотреть на примере.

```
for i in 'мама':  
    print (i)
```

Запустите эту программу и посмотрите, что получится.

Следующий графический пример поможет вам понять принцип действия оператора *for*. Эта программа построит нам концентрические окружности, центр которых совпадает с центром экрана. Расстояние между окружностями (шаг по радиусу) равно 20 пикселям.

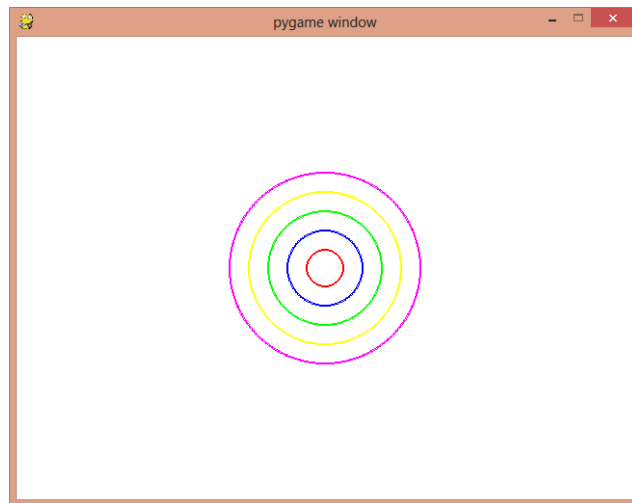


```

import pygame
pygame.init()
from pygame.color import THECOLORS
colors=["red","blue","green","yellow","magenta"]
screen = pygame.display.set_mode([640,480])
screen.fill([255,255,255]) # fill screen with white
for i in range (5,0,-1):
    pygame.draw.circle(screen, THECOLORS[colors[i-1]], [320,240], i*20, 0)
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()

```

В результате получается следующий рисунок:



Рассмотрим теперь такую задачу: нарисовать разноцветные закрашенные кольца. Алгоритм здесь следующий: рисуем закрашенный круг. Затем концентрический с ним закрашенный круг другого цвета и меньшего радиуса. И так далее. При этом необходимо, чтобы счетчик цикла уменьшался от максимального значения к минимальному. Для этих целей в языке Python предусмотрен так называемый нисходящий цикл. Вот как с его помощью решается предложенная задача:

```

import pygame
pygame.init()
from pygame.color import THECOLORS
colors=["red","blue","green","yellow","magenta"]
screen = pygame.display.set_mode([640,480])
screen.fill([255,255,255]) # fill screen with white
for i in range (5,0,-1):
    pygame.draw.circle(screen, THECOLORS[colors[i-1]], [320,240], i*20, 0)
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():

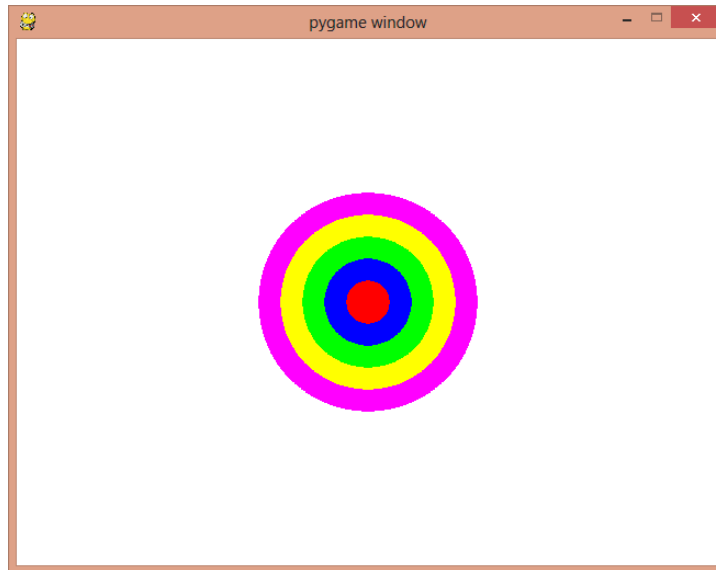
```

```

    if event.type == pygame.QUIT:
        running = False
pygame.quit()

```

В результате получаем:



### Задания для самостоятельной работы

1. Нарисуйте с помощью цикла российский флаг.
2. Нарисуйте радугу.
3. Постройте правильный многоугольник. Количество углов вводится с клавиатуры.
4. Нарисуйте на экране циферблат часов

#### 1.6.2 Циклы с условием

Цикл `for` замечательно работает, если вы заранее знаете, сколько итераций вам требуется сделать. Но иногда нужно, чтобы цикл выполнялся до наступления некоего события, и количество итераций в этом случае заранее оценить просто нереально. И здесь вам на помощь приходит цикл `while`.

Условный цикл в Python реализуется при помощи ключевого слова `while`. Пример программы с таким циклом приведен ниже. Запустите эту программу, чтобы посмотреть, как она работает.

```

print ("Type 3 to continue, anything else to quit.")
someInput = input()

while someInput == '3':
    print ("Thank you for the 3. Very kind of you.")
    print ("Type 3 to continue, anything else to quit." )
    someInput = input()
print ("That's not 3, so I'm quitting now.")

```

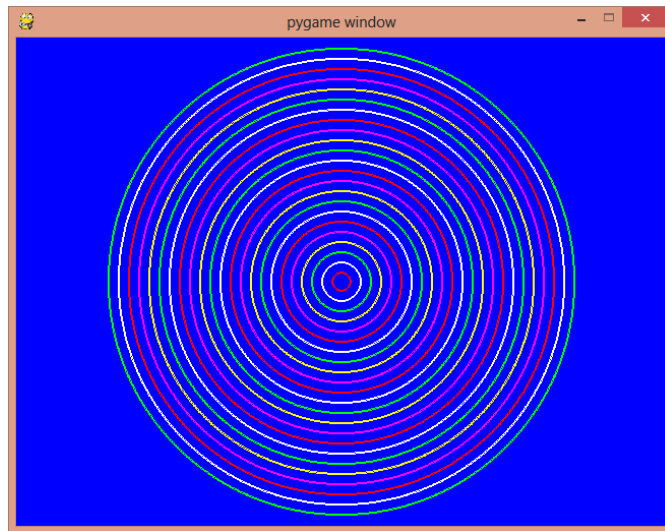
Эта программа постоянно просит пользователя ввести данные. *Пока* пользователь вводит цифру 3, условие соблюдается и цикл продолжается. Ввод любого *другого* символа нарушает условие продолжения цикла, и цикл завершается.

Следующий пример также поясняет работу цикла с условием.

```
#Сколько простых чисел в интервале от n до 1
print('Считаем количество простых чисел, меньших n')
n=int(input('Введите целое число\n'))
s=0
while n>1:
    f=True
    k=n//2
    for i in range(2,k+1):
        if (n%i)==0:
            f=False
    if f:
        print(n)
        s=s+1
    n=n-1
print('В интервале',s,'простых чисел')
input()
```

Перейдем к графическим примерам. Следующая программа с помощью цикла while нарисует окружности, диаметр которых ограничен высотой экрана на синем фоне.

```
import pygame, sys
pygame.init()
from pygame.color import THECOLORS
colors=["red","white","green","yellow","magenta"]
screen = pygame.display.set_mode([640,480])
screen.fill([0,0,255]) # fill screen with white
r=10
col=0
while r<240:
    pygame.draw.circle(screen, THECOLORS[colors[col]], [320,240], r, 2)
    r=r+10
    col=col+1
    if col==5:
        col=0
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```



### Задания для самостоятельной работы

1. Подсчитать количество четных и нечетных цифр в целом числе.
2. Сколько чисел натурального ряда нужно взять, чтобы сумма не превышала  $b$
3. Создать программу, реализующую алгоритм Евклида – алгоритм нахождения наибольшего общего делителя (НОД) пары целых чисел
4. Попадает круг вверх, вниз и на границу. Решить задачу с помощью цикла `while`.
5. Два круга: попадает ли один в другой. Решить задачу с помощью цикла `while`.
6. Два прямоугольника: попадает ли один в другой. Решить задачу с помощью цикла `while`.

#### 1.6.3 Неуправляемое движение объекта по экрану

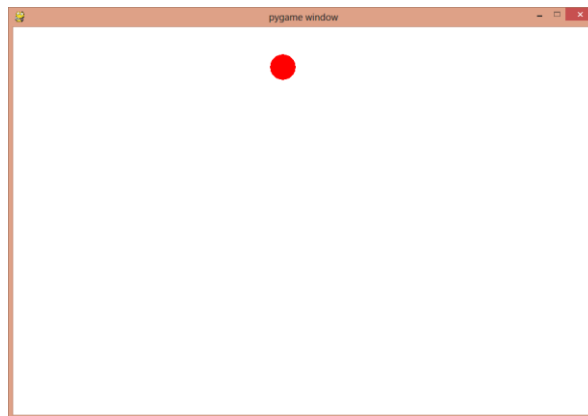
Следующая программа реализует неуправляемое движение объекта (круг) по экрану. Этот необходимый для динамических игр элемент реализуется с помощью цикла `while`. Круг движется по графическому окну и отскакивает от его краев. Необходимые комментарии вы найдете в тексте программы.

```
#Неуправляемое движение
import pygame
pygame.init()
from pygame.color import THECOLORS
screenX=900 #размеры графического окна
screenY=600
screen = pygame.display.set_mode([screenX,screenY])
x=640 #координаты начального положения круга
y=480
dx=2 #шаг по горизонтали
dy=2 #по вертикали
r=20 #радиус круга
pygame.draw.circle(screen, THECOLORS['red'], [x,y], r, 0)
running = True
while running:
    screen.fill(THECOLORS['white']) #стираем круг в предыдущем положении
    x=x+dx #перемещаем круг по x и по y
```

```

y=y+dy
pygame.draw.circle(screen, THECOLORS['red'], [x,y], r, 0) # рисуем круг в новом
#положении
if (x<=r) or (x>=screenX-r): # проверка на отражение от краев окна
    dx=-dx # отражение от края окна
if (y<=r) or (y>=screenY-r):
    dy=-dy
pygame.time.delay(10) # задержка на 10 миллисекунд
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
pygame.display.flip()
pygame.quit()

```



## 1.7 События

В этом разделе вы познакомитесь с дополнительным инструментом ввода – *событиями* (events). При этом мы детально рассмотрим, что делает код закрытия Pygame-окна и как он функционирует. Также вы узнаете, каким образом ввести данные при помощи мыши и научите свои программы немедленно реагировать на нажатие клавиш.

На вопрос «Что такое событие?» вы, скорее всего, ответите: «То, что происходит». Это определение работает не только в реальном мире, но и в программировании. У программиста возникает необходимость отреагировать на «происходящее», в частности на:

- перемещение указателя мыши или щелчок мыши;
- нажатие клавиш;
- завершение определенного временного интервала.

Большинство написанных нами программ от начала до конца работали по совершенно предсказуемому алгоритму, отвлекаясь разве что на циклы или условия. Но существует и другой класс программ, которые называют *событийно-управляемыми* (event-driven), и они работают немного по-другому. Обычно они не выполняют никаких действий, ожидая, пока не случится определенное *событие*. А после этого начинают процедуру его *обработки*.

Чтобы событийно-управляемые программы могли «увидеть» происходящее событие, они должны его «ждать». Программа постоянно сканирует ту часть компьютерной памяти, которая обычно подает сигнал о возникновении события. Во время ее работы это происходит снова и снова. Наступление события отслеживается при помощи цикла. Он называется *циклом событий* (event loop).

Все Pygame-программы, которые мы писали, заканчивались циклом while. Было отмечено, что этот цикл функционирует, пока работает программа. Именно он является циклом событий.

Все эти события наступают после перемещения указателя мыши, щелчка кнопки мыши или нажатия клавиши. Что происходит потом? Цикл событий постоянно сканирует определенный фрагмент памяти компьютера. Фрагмент памяти, в котором хранятся события, называется *очередью событий*. Очередь событий представляет собой список всех совершившихся событий, расположенных в порядке их возникновения

Программа должна знать, когда пользователь нажимает клавишу или двигает указатель. Все эти нажатия, щелчки и перемещения указателя мыши относятся к *событиям*, и программе нужно иметь информацию о том, что ей в этих случаях делать. Она должна их *обработать*. Часть программы, реагирующая на определенное событие, называется *обработчиком события*.

Обрабатываются не все события. При перемещении указателя по рабочему столу возникают сотни событий, ведь цикл событий выполняется быстро. Каждую долю секунды при смещении указателя мыши хотя бы на миллиметр в сторону генерируется новое событие. Но ваша программа может не обращать внимания на перемещения указателя. Значение могут иметь только щелчки в определенных местах. В этом случае программа будет игнорировать события mouseMove, обращая внимание только на события mouseClicked.

Событийно-управляемые программы имеют обработчики только нужных событий. Если движением корабля в игре игрок управляет при помощи клавиш со стрелками, нужно написать обработчик события keyDown. Если же движение корабля контролируется при помощи мыши, потребуется уже обработчик события mouseMove.

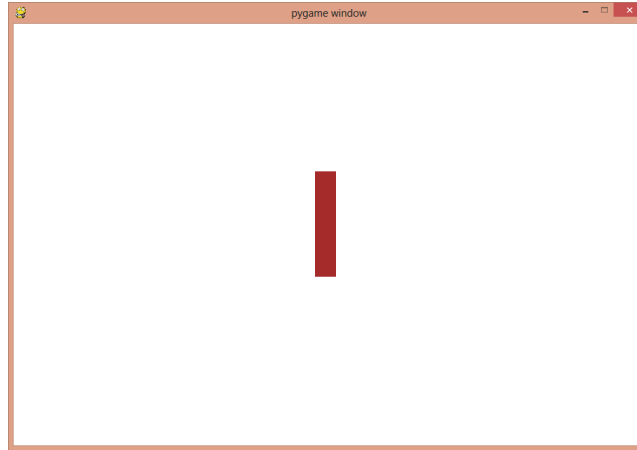
Рассмотрим события клавиатуры. Предположим, что после нажатия клавиши должно быть выполнено некое действие. В Pygame-модуле за это отвечает событие KEYDOWN. В качестве примера возьмем программу управляемого клавишами движения объекта по графическому окну.

```
#Управляемое движение
import pygame
pygame.init()
from pygame.color import THECOLORS
screenX=900
screenY=600
screen = pygame.display.set_mode([screenX,screenY])
xr=400
yr=200
width=30
height=150
step=10#Шаг ракетки (прямоугольник)
#Ракетка в начальном положении
pygame.draw.rect(screen,THECOLORS['brown'], [xr, yr, width, height], 0)
running = True
while running:
    screen.fill(THECOLORS['white'])
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:#Движение ракетки
            if event.key == pygame.K_UP:
                yr=yr-step
            elif event.key == pygame.K_DOWN:
```

```

        yr=yr+step
    elif event.key == pygame.K_LEFT:
        xr=xr-step
    elif event.key == pygame.K_RIGHT:
        xr=xr+step
    pygame.draw.rect(screen,THECOLORS['brown'], [xr, yr, width, height], 0)
    pygame.display.flip()
pygame.quit()

```



Метод `pygame.event.get()` получает список всех событий в очереди. Цикл `for` последовательно просматривает все события списка и, обнаружив событие `QUIT`, присваивает переменной `running` значение `False`. В результате цикл `while` и работа программы завершаются. Итак, мы выяснили, как работает код завершения работы программы при щелчке по кнопке закрытия Pygame-окна.

Познакомимся еще с одним типом событий. Так как нам нужно поймать момент нажатия клавиши, потребуется событие `KEYDOWN` (см. код выше). Именами `K_UP`, `K_DOWN`, `K_LEFT` и `K_RIGHT` Pygame обозначает клавиши стрелок.

При нажатии и удержании клавиши со стрелкой ракетка все равно перемещается всего на один шаг вверх или вниз. Дело в том, что мы не объяснили программе, как реагировать на *удержание* нажатой клавиши. В ответ на действие пользователя генерируется одно событие `KEYDOWN`, хотя в случае удержания клавиши Pygame-модуль позволяет генерировать множественные события `KEYDOWN`. Эта операция называется *повтором нажатия клавиши* (key repeat). Вы указываете, сколько времени должно пройти до момента повтора и как часто следует повторять операцию. Все параметры задаются в миллисекундах (сотых долях секунды). Вот как это выглядит:

```

delay = 100
interval = 50
pygame.key.set_repeat(delay, interval)

```

Параметр `delay` указывает модулю Pygame, через какой промежуток времени следует повторить действие, а параметр `interval` задает темп повтора, то есть промежуток между отдельными событиями `KEYDOWN`. Добавьте этот фрагмент кода в программу управляемого движения (после строки `pygame.init`, но перед циклом `while`) и посмотрите, как изменится поведение программы.

Теперь попробуем сделать так, чтобы мячом (круг) можно было управлять при помощи мыши. Это позволит вам познакомиться со способами обработки событий мыши и применения информации о положении указателя мыши.

Чаще всего используются следующие три события мыши:

- `MOUSEBUTTONUP`;
- `MOUSEBUTTONDOWN`;

- **MOUSEMOTION.**

Застави мяч следовать за перемещениями указателя в Pygame-окне. Это можно реализовать с помощью следующего кода:

```
#Перемещение с помощью мыши
import pygame, sys, random
pygame.init()
from pygame.color import THECOLORS
from pygame import font
screen = pygame.display.set_mode([640,480])
running = True
r=30
x=320
y=240
my_down=False #признак нажатия кнопки мыши
while running:
    screen.fill(THECOLORS['white'])
    pygame.draw.circle(screen, THECOLORS['red'],[x,y], r, 0)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.MOUSEBUTTONDOWN:
            my_down = True #Нажали кнопку
        elif event.type == pygame.MOUSEBUTTONUP:
            my_down = False #Отпустили кнопку
        elif event.type == pygame.MOUSEMOTION:
            if my_down: #Если кнопка нажата,
                (x1,y1)= event.pos #Определяем положение курсора мыши
                if (abs(x1-x)<r) and (abs(y1-y)<r):#Если курсор внутри круга
                    x=x1 #Перемещаем мяч
                    y=y1
    pygame.display.flip()
pygame.quit()
```

Мяч должен перемещаться в случае, если нажата кнопка мыши. За это отвечает переменная *my\_down*. При событии *MOUSEBUTTONDOWN* она становится *True*, при событии *MOUSEBUTTONUP* – *False*. Перемещение (событие *MOUSEMOTION*) происходит, если *my\_down* равна *True*. Функция *event.pos* возвращает координаты курсора мыши. Остальные действия понятны из комментариев к программе.

## 1.8 Функции

Функция – небольшая программа, выполняющая определенную задачу. Ниже речь идет о создании собственных (нестандартных) функциях.

Ниже мы определим функцию, которую затем используем. Функция выводит на экран почтовый адрес.

```
def printMyAddress():
    print ("Leonid Popov")
    print ("630128 Novosibirsk")
```



```
printMyAddress()
```

Получается следующий результат:

```
>>>  
Leonid Popov  
630128 Novosibirsk
```

*Вызов* функции означает запуск ее внутреннего кода. Если вы определите функцию, но ни разу ее не вызовете, ее код так и не будет запущен. Для вызова функции достаточно указать ее имя, за которым следуют скобки.

В программировании термин *аргумент* означает передаваемый в функцию фрагмент информации. Это называется *передачей* аргумента в функцию. Представьте, что вам нужна функция, выводящая на печать адреса всех членов вашей семьи. Адрес во всех случаях будет один и тот же, а вот имена разные. В уже готовой программе имя «Leonid Popov» запрограммировано в коде функции, но его можно превратить в переменную. И эта переменная будет передаваться в функцию в момент ее вызова.

Как это работает, проще всего продемонстрировать на примере. В приведенной ниже программе функция вывода адреса была отредактирована таким образом, что имя адресата превратилось в аргумент. Аргументы, как и все другие переменные, имеют имена. Эту переменную назовем *myName*. После запуска функции переменная *myName* принимает значение переданного в функцию аргумента. Этот аргумент указывается в скобках в момент вызова функции.

В нашей программе аргументу *myName* присваивается значение "Leonid Popov".

```
def printMyAddress(myName):  
    print (myName)  
    print ("630128 Novosibirsk")
```

```
printMyAddress("Leonid Popov")
```

Получается:

```
Leonid Popov  
630128 Novosibirsk
```

Это напоминает результат работы первой программы, в которой аргументы не применялись. Но теперь мы можем менять имя адресата при каждом вызове функции.

Наша функция обладает одним аргументом. Но аргументов может быть и несколько. Более того, их количество ничем не ограничено. Рассмотрим следующий пример:

```
def printMyAddress(someName, houseNum):  
    print (someName)  
    print (houseNum)
```

```
printMyAddress("Leonid Popov", "45")  
printMyAddress("Ivan Petrov", "64")
```

Эта функция с двумя аргументами печатает не только имя, но и номер дома.

С помощью аргументов можно передать в функцию информацию, но функция может и сама возвращать информацию в вызывающий код. Эта информация называется *результатом* или *возвращаемым значением* функции.

Давайте напишем программу с возвращающей значение функцией. В нашем примере возвращает значение функция `dengi_tovar()`. Количество денег и цену товара. Это значение сохраняется в переменной `kol_pepsi`. Поэтому если раньше мы ограничивались именем функции, то теперь нам понадобится переменная, знак присваивания (`=`) и только после этого имя функции. То есть мы присвоим переменной результат, возвращаемый функцией `dengi_tovar()`

```
#Эта программа поможет контролировать ваши расходы
print('Программа вычисляет количество приобретенного товара')
dengi=int(input('Сколько у вас денег?\n'))
def dengi_tovar(dengi,zena):
    kolvo=dengi//zena
    return(kolvo)
kol_peps=dengi_tovar(dengi,5)
print('Вы купили',kol_peps,'бутылок пепси')
```

В нашем примере переменная `zena` используется только внутри функции. В этом случае говорят о том, что *областью видимости* переменной `zena` является функция `dengi_tovar()`. Еще они называются *локальными переменными*. Итак, `zena` – это локальная переменная функции `dengi_tovar()`. Имена внутри функций создаются только после ее запуска. До запуска и после его завершения они не существуют. В интерпретатор Python встроен механизм *управления памятью*, который выполняет все эти действия автоматически. Интерпретатор Python после запуска функции создает новое имя, которое *удаляется после того, как функция заканчивает свою работу*. Последнее крайне важно: после завершения функции все использовавшиеся внутри нее имена перестают существовать.

В процессе работы функции *внешние* имена как бы на время блокируются. Используются только имена, определенные внутри функции. Часть программы, в которой функционирует (или потенциально может функционировать) переменная, называется *областью видимости* (`scope`).

В отличие от *локальной* переменной `zena`, переменная `dengi` определена *не внутри* функции, а в основной части программы. Переменные с более широкой областью видимости называются *глобальными*

При попытке изменить внутри функции значение *глобальной переменной* интерпретатор Python создает вместо нее новую *переменную – локальную*. Это сделано, чтобы исключить возможность случайного изменения глобальной переменной внутри функции. Но бывают случаи, когда в подобных обстоятельствах нам *требуется* изменить глобальную переменную. Как это можно сделать? В Python есть ключевое слово `global`, позволяющее обойти запрет на изменение глобальной переменной. Вот как оно используется:

```
def dengi_tovar(dengi,zena):
    global kolvo
```

Ключевое слово `global` позволяет интерпретатору Python не создавать локальную переменную `kolvo`, а воспользоваться глобальной переменной `kolvo`. Если глобальной переменной с таким названием не существует, он ее создаст.

Рассмотрим еще один пример: создадим функцию возведения числа в целую степень и функцию, вычисляющую котангенс. Первый вариант программы будет использовать глобальные переменные:

```
#Возведение числа в степень (глобальные переменные)
from math import*
print('Программа возводит число в степень')
n=float(input('Введите число\n'))
deg=int(input('Введите степень\n'))
def degree(n,deg):
    global res
    res=1.0
    i=0
    while i<abs(deg):
        res=res*n
        i=i+1
    if deg<0: res=1/res
def ctg(x):
    global ctan
    ctan=cos(x)/sin(x)
degree(n,deg)
print('%.4f'%res)
ctg(pi/3)
degree(ctan,3)
print('%.4f'%res)
input()
```

Результат работы программы:

```
>>>
Программа возводит число в степень
Введите число
2
Введите степень
2
4.0000
0.1925

>>>
```

Вариант с функцией return()

```
#Возведение числа в степень и котангенс
from math import*
print('Программа возводит число в степень')
n=float(input('Введите число\n'))
deg=int(input('Введите степень\n'))
def degree(n,deg):
    res=1
    i=0
    while i<abs(deg):
```

```

        res=res*n
        i=i+1
        if deg<0: res=1/res
        return(res)
res=degree(n,deg)
def ctg(x):
    ctan=cos(x)/sin(x)
    return(ctan)
print("%.4f"%res)
res=degree(ctg(pi/4),-1)
print("%.4f"%res)
input()

```

Результат:

```

Программа возводит число в степень
Введите число
5
Введите степень
-2
0.0400
1.0000

```

Следует отметить, что в современном программировании предпочтительным считается вариант с инструкцией return. Поэтому в дальнейших примерах мы будем использовать именно этот подход.

#### 1.8.1 Совмещение неуправляемого и управляемого движений

Эту задачу следует решать, используя функции Python. Ниже приведен код такой программы:

```

#Неуправляемое и управляемое движение
import pygame, sys, random
from pygame import font
pygame.init()
from pygame.color import THECOLORS
font = pygame.font.Font(None, 24)
screenX=900
screenY=600
screen = pygame.display.set_mode([screenX,screenY])
x=40
y=480
dx=2
dy=2
r=20
xr=750
yr=200
width=30
height=200
step=10
n=0

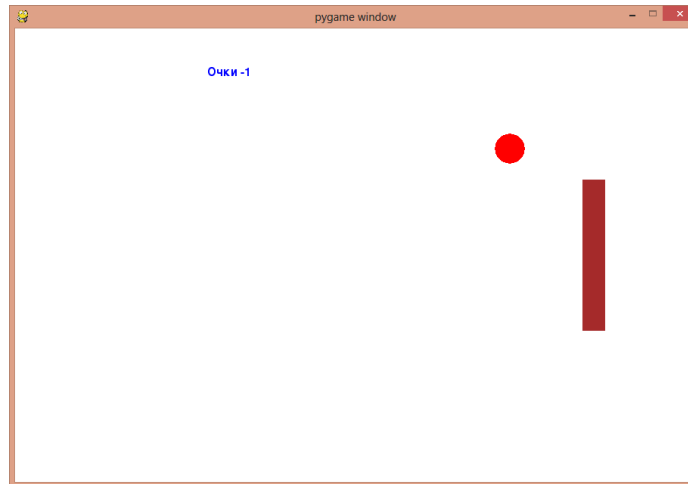
```

```

color=THECOLORS['red']
def schet(n)#Счет очков
    text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    return(text)
def move_ball(x,y,dx,dy,n,r,color,text):#Неуправляемое движение шарика
    x=x+dx
    y=y+dy
    pygame.draw.circle(screen,color,[x,y], r, 0)
    if (x>=screenX-r):
        x=40
        y=random.randint(10, 400)
        n=n-2
        text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    if (x<=r):
        dx=-dx
    if (y<=r) or (y>=screenY-r):
        dy=-dy
    return(x,y,dx,dy,n,text)
def proverka(x,y,xr,yr,width,height,dx,n,text):#Проверка столкновения шарика и
ракеты
    if (y>=(yr-r))and (y<=(yr+height))and (x>=(xr-r))and (x<(xr+width)):
        dx=-dx
        n=n+1
        text=schet(n)
        return(dx,n,text)
def move_racet(yr,step):#Движение ракеты
    if event.key == pygame.K_UP:
        yr=yr-step
    elif event.key == pygame.K_DOWN:
        yr=yr+step
    return(yr)
#Шарик и ракета в начальном положении
pygame.draw.circle(screen, color,[x,y], r, 0)
pygame.draw.rect(screen,THECOLORS['brown'], [xr, yr, width, height], 0)
text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
delay = 100
interval = 50
pygame.key.set_repeat(delay, interval)
running = True
while running:
    screen.fill(THECOLORS['white'])
    screen.blit(text, [250, 50] )
    x,y,dx,dy,n,text=move_ball(x,y,dx,dy,n,r,color,text)
    dx,n,text=proverka(x,y,xr,yr,width,height,dx,n,text)
    pygame.time.delay(10)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            yr=move_racet(yr,step)
    pygame.draw.rect(screen,THECOLORS['brown'], [xr, yr, width, height], 0)
    pygame.display.flip()

```

`pygame.quit()`



## 1.9 Модули

*Модулем (module)* называется часть, или фрагмент, какого-то элемента. Если элемент представлен в виде набора фрагментов или может быть легко разобран на фрагменты, его называют *модульным (modular)*. Превосходной иллюстрацией модульной конструкции являются блоки для игры LEGO. Из массы различных фрагментов можно создать множество разнообразных конструкций. В Python модулями называются небольшие фрагменты большой программы. Каждый модуль находится в отдельном файле на жестком диске вашего компьютера. Можно взять большую программу и разбить ее на несколько модулей, или файлов. А можно пойти другим путем – начать с маленького модуля и, добавляя фрагменты, создать большую программу.

Давайте создадим модуль.

```
# это файл "my_module.py"
# мы собираемся использовать его в другой программе
# Программа по количеству введенных секунд определяет сколько это минут и секунд
def min_sec(sec):
    minut=sec//60
    sec=sec%60
    return(minut,sec)
minut,sec=min_sec(sec)
print("Это",minut,'минут и',sec,'секунд')
input()
```

Готово! Вы только что создали модуль! Он содержит единственную функцию `min_sec()`, выполняющую преобразование количество секунд в минуты и секунды.

Теперь воспользуемся модулем `my_module.py` в другой программе. Если вы хотите воспользоваться заключенной в модуле функциональностью, первым делом следует объяснить интерпретатору Python, какой модуль вам нужен. Для включения в программу сторонних модулей в Python существует ключевое слово `import`. Вот как оно используется:

```
import my_module
```

Давайте напишем программу, в которой будет фигурировать только что созданный нами модуль. Мы собираемся выполнить преобразование температуры при помощи функции `min_sec()`. Вы уже умеете пользоваться функциями и передавать в них параметры (или аргументы). Но в данном случае функция находится в отдельном файле, не связанном

с основной программой, поэтому нам потребуется ключевое слово `import`. Представленная программа использует созданный нами модуль `my_module.py`.

```
import my_module
print('По количеству секунд найти количество минут и секунд')
sec=int(input('Введите количество секунд\n'))
minut,sec=mymodule.min_sec(sec)
print("Это',minut,'минут и',sec,'секунд')
input()
```

Импортировать функции и объекты из модуля можно и по другому:

```
import my_module
from my_module import min_sec(sec)
# или from my_module * - импортируются все имена в my_module
sec=int(input('Введите количество секунд\n'))
minut,sec=mymodule.min_sec(sec)
print("Это',minut,'минут и',sec,'секунд')
input()
```

Интерпретатор Python поставляется с массой стандартных модулей, позволяющих в числе прочего выполнять такие операции, как поиск файлов, информирование о времени (или отсчет времени) и генерация случайных чисел. Иногда говорят, что Python обладает «встроенным арсеналом», подразумевая именно стандартные модули. Они известны как *стандартная Python-библиотека* (Python Standard Library).

Почему эти операции помещены в отдельные модули? На самом деле без этого можно было обойтись, но разработчики языка Python решили, что это повысит эффективность работы. В противном случае вам пришлось бы включать в каждую Python-программу все возможные функции. А так вы можете обойтись только теми, которые вам требуются.

Создадим модуль `upr_neupr_udar`. От предыдущей программы он отличается тем, что цикл `while` включен в функцию `move_racet()`.

```
import pygame, sys, random
from pygame import font
pygame.init()
from pygame.color import THECOLORS
font = pygame.font.Font(None, 24)
screenX=900
screenY=600
screen = pygame.display.set_mode([screenX,screenY])
x=40
y=480
dx=2
dy=2
r=20
xr=750
yr=200
width=30
height=200
step=10
n=0
```

```

color=THECOLORS['red']
def schet(n):
    text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    return(text)
def proverka(x,y,xr,yr,width,height,dx,n,text):
    if (y>=(yr-r))and (y<=(yr+height))and (x>=(xr-r))and (x<(xr+width)):
        dx=-dx
        n=n+1
        text=schet(n)
    return(dx,n,text)
def move_ball(x,y,dx,dy,n,r,color,text):
    x=x+dx
    y=y+dy
    pygame.draw.circle(screen,color,[x,y], r, 0)#Движение шарика
    if (x>=screenX-r):
        x=40
        y=random.randint(10, 400)
        n=n-2
        text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    if (x<=r):
        dx=-dx
    if (y<=r) or (y>=screenY-r):
        dy=-dy
    dx,n,text=proverka(x,y,xr,yr,width,height,dx,n,text)
    return(x,y,dx,dy,n,text)

def move_racet(xr,yr,x,y,dx,dy,step,n):
    pygame.draw.circle(screen, color,[x,y], r, 0)
    pygame.draw.rect(screen,THECOLORS['brown'] , [xr, yr, width, height], 0)
    text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    delay = 100
    interval = 50
    pygame.key.set_repeat(delay, interval)
    running = True
    while running:
        screen.fill(THECOLORS['white'])
        screen.blit(text, [250, 50] )
        x,y,dx,dy,n,text=move_ball(x,y,dx,dy,n,r,color,text)
        pygame.time.delay(10)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_UP:
                    yr=yr-step
                elif event.key == pygame.K_DOWN:
                    yr=yr+step
        pygame.draw.rect(screen,THECOLORS['brown'] , [xr, yr, width, height], 0)
        pygame.display.flip()
    return(yr)

```

И вот как будет теперь выглядеть программы, использующая этот модуль.



```
#Неуправляемое и управляемое движение
import pygame, sys, random, mod_udar_1
from pygame import font
from mod_udar import*
pygame.init()
step=10
yr=mod_udar.move_racet(xr,yr,x,y,dx,dy,step,n)
pygame.quit()
```

В дальнейшем мы обязательно будем использовать эту технологию

## 1.10 Списки и словари

Как вы уже знаете, Python позволяет сохранять в памяти различные значения и извлекать их оттуда с помощью имен. До этого момента мы сохраняли *строки* и *числа* (как *целые*, так и с *плавающей точкой*). Но иногда необходимо сохранить целую группу, или *коллекцию* (collection), элементов. Это позволяет оперировать всей коллекцией одновременно и упрощает отслеживание групп элементов. Одной из разновидностей коллекции является *список* (list), другой – *словарь* (dictionary). Именно о списках и словарях пойдет речь в этой главе. Вы узнаете, как они создаются, модифицируются и применяются в программах.

Списки крайне полезны и фигурируют во множестве программ. Они будут часто появляться в примерах кода в следующих главах. Ведь многие графические объекты в играх хранятся именно в виде списков.

Представим себе, что вам нужно создать список студентов вашей группы. На языке Python это будет выглядеть так:

```
Students=['Petrov', 'Ivanova', 'Семенов']
```

А если попросить вас написать свои счастливые числа, может получиться нечто такое:

```
luckyNumbers = [2, 7, 14, 26, 30]
```

В данном случае как students, так и luckyNumbers — примеры Python-списков, а входящие в них отдельные объекты называются *элементами* (items). Как видите, Python-списки не сильно отличаются от привычных для вас реальных списков. Квадратные скобки показывают начало и конец списка, а запятые отделяют элементы друг от друга.

Чтобы создать список, как и любую другую переменную, используем присваивание некоего значения. Мы это уже видели на примере переменной luckyNumbers. Кроме того, можно создать пустой список:

```
newList = []
```

В квадратных скобках нет ни одного элемента, то есть список является пустым. Но какая польза от пустого списка? Зачем он может кому-то понадобиться? Дело в том, что часто возникает ситуация, когда вы заранее не знаете, что именно попадет в список. Неизвестны ни количество элементов, ни их состав. Известно только, что вы будете пользоваться списком. А если у вас есть пустой список, туда можно добавить что угодно. Давайте посмотрим, как это делается.

Для добавления элементов в список служит метод append()

```
>>> friends = []
>>> friends.append('Иван')
>>> print (friends)
```

Вот какой результат вы получите:

```
['Иван']
```

Попробуйте добавить еще один элемент:

```
>>> friends.append('Маша')
>>> print friends
```

Результат:

```
['Иван', 'Маша']
```

Имейте в виду, что добавлять элементы вы можете только в уже существующий список (пустой или нет). Поэтому первым делом его нужно создать.

Почему между словами `friends` и `append()` стоит точка? Это первый предвестник большой темы, посвященной объектам. *Объектами* (objects) являются многие элементы языка Python. Для выполнения какого-либо действия с объектом нужно указать его имя, поставить точку и добавить наименование операции.

Списки могут содержать любые данные, которые можно сохранять в Python. В этот перечень входят числа, строки, объекты и даже другие списки. Элементы списка не обязаны принадлежать к одному типу. Это означает, что в одном списке могут одновременно содержаться, например, числа и строки. Это будет выглядеть примерно так:

```
my_list = [5, 10, 23.76, 'Привет', myTeacher, 7, another_list]
```

Давайте создадим список с простыми элементами, например, с буквами латинского алфавита, чтобы при изучении свойств списков было проще понять, что именно происходит.

```
letters = ['a', 'b', 'c', 'd', 'e']
```

Доступ к отдельному элементу списка происходит по номеру его индекса. Индексы начинаются с 0, поэтому первым элементом нашего списка, то есть элементом `letters[0]`, будет буква *a*:

```
>>> print (letters[0])
a
```

Попробуем еще раз:

```
>>> print (letters[3])
d
```

Индексы позволяют обеспечить доступ сразу к нескольким элементам списка. Эта операция называется *получением среза* (slicing) списка:

```
>>> print (letters[1:4])
```

## Результат

`['b', 'c', 'd']`

Как и в случае с функцией `range()` в цикле `for`, при получении среза извлечение элементов начинается с первого указанного индекса и заканчивается *перед* вторым. Именно поэтому в предыдущем примере мы получили три, а не четыре элемента. Запомните, что получаемое в результате среза количество элементов равно разнице между индексами ( $4 - 1 = 3$ , то есть мы получаем три элемента).

Есть еще один момент, который нужно учитывать при получении среза списков: в результате создается другой список (обычно меньшего размера). Этот меньший список называют *срезом* (slice) исходного, который при этом остается неизменным. Срез является частичной *копией* оригинала. Посмотрите, какие разные результаты получаются в этом фрагменте кода:

```
>>> print (letters[1])
b
>>> print (letters[1:2])
['b']
```

В первом случае нам показывают элемент списка. Во втором нам возвращают список с одним элементом. Кажется, что разница совсем невелика, но о ней нужно помнить. Для получения *элемента* списка указывается его индекс. А для получения одноэлементного среза списка через *двоеточие* указываются индексы соседних элементов.

Чтобы лучше понять разницу, запустите следующий фрагмент кода:

```
>>> print (type(letters[1]))
<type 'str'>
>>> print (type(letters[1:2]))
<type 'list'>
```

Мы вывели на экран тип (`type`) каждого элемента и убедились, что в первом варианте мы получаем сам элемент, то есть в данном случае строку (`str`), а во втором — список (`list`). Меньший список, создаваемый в результате получения среза, представляет собой копию фрагмента исходного списка. Вы можете его редактировать, и это никак не повлияет на исходный список.

Для получения срезов существуют сокращенные варианты записи кода. Нельзя сказать, чтобы они сильно сэкономили время, но программисты, как правило, люди ленивые и предпочитают не тратить усилия понапрасну. Поэтому вы должны знать о существующих сокращениях, чтобы распознать их в чужом коде и понять, что происходит. Это очень важное умение, так как именно анализ чужого кода помогает при освоении нового языка программирования в частности и программирования в целом.

Если срез должен начинаться с первого элемента списка, поставьте двоеточие и укажите нужное количество элементов. Вот так:

```
>>> print (letters[:2])
['a', 'b']
```

Обратите внимание на отсутствие цифры перед двоеточием. При такой записи вы получите все элементы с начала списка до элемента с указанным вами индексом (но не включая этот элемент). Аналогичным способом делается срез конца списка:

```
>>> print (letters[2:])  
['c', 'd', 'e']
```

Число, за которым следует двоеточие, даст вам фрагмент от элемента с указанным индексом до конца списка. Можно вообще не указывать никаких чисел, поместив в скобки только двоеточие. В этом случае вы получите список целиком:

```
>>> print (letters[:])  
['a', 'b', 'c', 'd', 'e']
```

Индекс дает нам доступ к изменению отдельных элементов списка:

```
>>> print (letters)  
['a', 'b', 'c', 'd', 'e']  
>>> letters[2] = 'z'  
>>> print (letters)  
['a', 'b', 'z', 'd', 'e']
```

Однако индекс невозможно использовать для добавления в список новых элементов. Например, пусть у нас есть список из пяти элементов с индексами от 0 до 4. При этом выполнить вот такую операцию нельзя:

```
letters[5] = 'f'
```

Это просто не сработает. Это все равно что пытаться изменить вещь, которой еще не существует. О способах добавления элементов в список мы поговорим в следующем разделе, но сначала вернем наш список к исходному виду:

```
>>> letters[2] = 'c'  
>>> print (letters)  
['a', 'b', 'c', 'd', 'e']
```

Операции над списками

Сложение списков

```
>>> a=[1,2,3]  
>>> b=[4,5,6]  
>>> a+b  
[1, 2, 3, 4, 5, 6]
```

Умножение

```
a*3  
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Методы добавления элементов

Вы уже знаете, что добавить элемент в список можно методом `append()`. Но существуют другие методы. Всего их три: `append()`, `extend()` и `insert()`.

Метод `append()` добавляет один элемент в конец списка.

Метод `extend()` добавляет несколько элементов в конец списка.

Метод `insert()` добавляет один элемент в произвольное место списка, необязательно в конец. Место вы выбираете сами.

С методом `append()` вы уже знакомы. Он добавляет в конец списка один элемент:

```
>>> letters.append('n')
>>> print (letters)
['a', 'b', 'c', 'd', 'e', 'n']
Добавим еще один элемент:
>>> letters.append('g')
>>> print (letters)
['a', 'b', 'c', 'd', 'e', 'n', 'g']
```

Обратите внимание на то, что буквы не упорядочены. Дело в том, что новые буквы метод `append()` помещает в самый конец. Если вам нужен упорядоченный список, следует прибегнуть к сортировке. С приемами сортировки вы познакомитесь чуть позже.

Метод `extend()` добавляет в конец списка несколько элементов:

```
>>> letters.extend(['p', 'q', 'r'])
>>> print (letters)
['a', 'b', 'c', 'd', 'e', 'n', 'g', 'p', 'q', 'r']
```

Обратите внимание: аргументом метода `extend()` является список. А так как список заключается в квадратные скобки, после слова `extend` указываются как обычные, так и квадратные скобки. Все содержимое списка, переданного вами в метод `extend()`, добавляется в конец оригинального списка.

Метод `insert()` добавляет единичный элемент в произвольное место списка. Место вставки указываете вы:

```
>>> letters.insert(2, 'z')
>>> print (letters)
['a', 'b', 'z', 'c', 'd', 'e', 'n', 'g', 'p', 'q', 'r']
```

Мы добавили букву `z` в позицию с индексом 2. В списке этот индекс соответствует третьей слева позиции (так как отсчет начинается с нуля). Буква `c`, изначально находившаяся в третьей позиции, сдвигается вправо и оказывается четвертой. Все прочие элементы списка также сдвигаются на одну позицию вправо.

Иногда методы `append()` и `extend()` выглядят одинаково, но функционируют они поразному. Вернемся к нашему исходному списку. Для начала воспользуемся методом `extend()`:

```
>>> letters = ['a','b','c','d','e']
>>> letters.extend(['f', 'g', 'h'])
>>> print (letters)
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

Теперь попробуем сделать то же самое методом `append()`:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> letters.append(['f', 'g', 'h'])
>>> print (letters)
```

```
['a', 'b', 'c', 'd', 'e', ['f', 'g', 'h']]
```

Что произошло? Как мы уже говорили, метод `append()` добавляет в список только один элемент. Как же получилось, что он добавил три элемента? Он этого не делал. Он добавил один элемент, который представляет собой список, содержащий три элемента. Поэтому внутри итогового списка появились дополнительные квадратные скобки. Напоминаем, что в список можно включать элементы любых типов, в том числе другие списки. Именно это мы и видим в данном случае.

Метод `insert()` отличается от метода `append()` только тем, что вставляет новый элемент в указанное вами место, в то время как метод `append()` всегда добавляет его в конец списка.

Что делать, если вам нужно удалить элемент из списка? Для этого существует три метода: `remove()`, `del` и `pop()`.

Метод `remove()` удаляет из списка выбранный вами элемент:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> letters.remove('c')
>>> print (letters)
['a', 'b', 'd', 'e']
```

Вам не нужно знать, где именно в списке находится удаляемый элемент. Достаточно, чтобы он там был. Попытка удаления несуществующего элемента приводит к сообщению об ошибке:

Рассмотрим остальные способы удаления элементов.

Метод `del` удаляет элемент по его индексу. Вот так:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> del letters[3]
>>> print (letters)
['a', 'b', 'c', 'e']
```

В данном случае мы удалили четвертый элемент (с индексом 3), то есть букву d.

Метод `pop()` извлекает из списка последний элемент и возвращает его вам. После этого ему можно присвоить имя:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> lastLetter = letters.pop()
>>> print (letters)
['a', 'b', 'c', 'd']
>>> print (lastLetter)
e
```

Метод `pop()` можно использовать и с индексом:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> second = letters.pop(1)
>>> print (second)
b
>>> print (letters)
['a', 'c', 'd', 'e']
```

В данном случае мы извлекли второй элемент (с индексом 1), то есть букву *b*. Элемент был сохранен в переменной *second* и одновременно удален из списка *letters*. Если в скобках ничего не указано, метод *pop()* дает вам последний элемент, одновременно удаляя его из списка. Если в скобках стоит какое-нибудь число, метод *pop(n)* выдает элемент с указанным индексом, удаляя его из списка.

Как найти в списке нужный элемент? При работе со списками часто возникает необходимость:

проверить, присутствует ли там некий элемент;  
определить его позицию в списке (то есть индекс).

Проверить, присутствует ли в списке определенный элемент, помогает ключевое слово *in*. Вот как это делается:

```
if 'a' in letters:
    print ("найден символ 'a' в списке letters")
else:
    print ("символ 'a' в списке letters не обнаружен")
```

Запись *'a' in letters* — часть *булевого*, или *логического*, выражения. Оно возвращает значение *True*, если в списке присутствует буква *a*, и значение *False* в противном случае.

Введите этот фрагмент кода в интерактивном режиме:

```
>>> 'a' in letters
True
>>> 's' in letters
False
```

Код сообщает нам, что в списке *letters* присутствует буква *a*, но нет буквы *s*. Теперь можно скомбинировать ключевое слово *in* с методом *remove()* и написать код, который не покажет сообщения об ошибке, даже если в списке нет нужного значения:

```
if 'a' in letters:
    letters.remove('a')
```

Этот код удаляет элемент из списка только в том случае, если он там присутствует. Местоположение элемента в списке определяет метод *index()*:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> print (letters.index('d'))
3
```

Теперь мы знаем, что буква *d* имеет индекс 3, то есть в списке она фигурирует на четвертой позиции. Подобно методу *remove()*, метод *index()*, не обнаружив в списке нужного элемента, выдает сообщение об ошибке, поэтому его тоже имеет смысл использовать вместе с ключевым словом *in*:

```
if 'd' in letters:
    print (letters.index('d'))
```

В начале своего знакомства с циклами вы видели, как цикл перебирает значения из списка. Еще вы узнали о функции *range()* и воспользовались ею для быстрой генерации списка перебираемых значений. Вы выяснили, что эта функция генерирует список чисел. Но

в цикле можно перебирать элементы любого списка, ограничиваться списком чисел вовсе не обязательно. Представьте, что нам нужно вывести на экран список букв по одной букве на строке. Для этого можно написать такой код:

```
>>> letters = ['a', 'b', 'c', 'd', 'e']
>>> for letter in letters:
print (letter)
a
b
c
d
e
```

На сей раз переменной цикла является letter. Цикл перебирает все значения из списка, и на каждой итерации рассматриваемый элемент сохраняется в переменной letter и выводится на экран.

Списки относятся к *упорядоченным* коллекциям. Это означает, что их элементы располагаются в определенном порядке и за каждым закреплено его место (индекс). Порядок размещения элементов сохраняется, пока вы не измените его методом insert(), append(), remove() или pop(). Но иногда после этих операций элементы оказываются не в том порядке, который вам нужен. А значит, перед дальнейшей работой со списком его требуется подвергнуть *сортировке*.

Для этого существует метод sort():

```
>>> letters = ['d', 'a', 'e', 'c', 'b']
>>> print (letters)
['d', 'a', 'e', 'c', 'b']
>>> letters.sort()
>>> print (letters)
['a', 'b', 'c', 'd', 'e']
```

Можно также добавить в метод sort() параметр, заставляющий выполнить сортировку в порядке убывания (от самого большого к самому маленькому):

```
>>> letters = ['d', 'a', 'e', 'c', 'b']
>>> letters.sort (reverse = True)
>>> print (letters)
['e', 'd', 'c', 'b', 'a']
```

Этот параметр, который называется reverse, делает именно то, что вам нужно, — инициирует сортировку списка в обратном порядке.

Существует еще один способ получения отсортированной копии без нарушения порядка размещения элементов в исходном списке. В Python для этой цели используется функция sorted(). Вот как она работает:

```
>>> original = [5, 2, 3, 1, 4]
>>> newer = sorted(original)
>>> print (original)
[5, 2, 3, 1, 4]
>>> print(newer)
[1, 2, 3, 4, 5]
```



Функция `sorted()` дает вам *отсортированную копию* исходного списка.

Иногда нам требуются списки, не допускающие изменения. Есть ли в Python подобные списки? Конечно же! Существует такой тип объектов, как *кортеж* (tuple), который является именно неизменным списком. Вот как он создается:

```
my_tuple = ("красный", "зеленый", "синий")
```

Здесь мы используем обычные, а не квадратные скобки, как было бы в случае списка. Так как кортежи не допускают изменения, их невозможно сортировать, в них нельзя добавить элемент или что-то из них удалить. После своего создания кортеж никак не меняется.

Иногда нам требуется *таблица (матрица)* из строк и столбцов (например, это отметки моих одноклассников по математике, естествознанию, чтению и правописанию). Как сохранить таблицу данных? Вы уже знаете, что для хранения нескольких элементов у нас служат списки. Поэтому отметки каждого ученика можно поместить в список:

```
>>> IvanMarks = [55, 63, 77, 81]
>>> PetrMarks = [65, 61, 67, 72]
>>> OlgaMarks = [97, 95, 92, 88]
```

Еще можно создать список для каждого предмета:

```
>>> mathMarks = [55, 65, 97]
>>> scienceMarks = [63, 61, 95]
>>> readingMarks = [77, 67, 92]
>>> spellingMarks = [81, 72, 88]
```

Но что делать, если вы хотите сохранить всю эту информацию в одной *структуре данных*? Чтобы создать единую структуру данных для оценок учеников по всем предметам, можно написать такой код:

```
>>> classMarks = [IvanMarks, PetrMarks, OlgaMarks]
>>> print (classMarks)
[[55, 63, 77, 81], [65, 61, 67, 72], [97, 95, 92, 88]]
```

Это даст вам список элементов, в котором каждый элемент также будет списком. То есть мы создаем список из других списков. Каждый элемент списка `classMarks` представляет собой список. Более того, список `classMarks` можно создать и без предварительного создания списков `joeMarks`, `tomMarks` и `bethMarks`. Вот как это делается:

```
>>> classMarks = [ [55,63,77,81], [65,61,67,72], [97,95,92,88] ]
>>> print (classMarks)
[[55, 63, 77, 81], [65, 61, 67, 72], [97, 95, 92, 88]]
```

Теперь выведем содержимое нашей структуры данных на экран. Список `classMarks` состоит из трех элементов, по одному для каждого ученика. Поэтому мы сможем просмотреть их с помощью цикла и ключевого слова `in`:

```
>>> for studentMarks in classMarks:
print (studentMarks)
[55, 63, 77, 81]
```

[65, 61, 67, 72]  
[97, 95, 92, 88]

Цикл обеспечил перебор списка classMarks. Переменная цикла в данном случае называется studentMarks. На каждой итерации цикла на экран выводится один элемент списка. Этот элемент представляет собой список с оценками конкретного ученика. (Списки учеников были созданы ранее.). Теперь у нас есть структура данных, в которой можно сохранить всю информацию одним блоком.

Как получить доступ к сохраненным в таблице (нашем списке списков) значениям? Мы уже знаем, что оценки первого ученика (joeMarks) хранятся в списке, который является первым элементом списка classMarks. Попробуем сделать так:

```
>>> print(classMarks[0])  
[55, 63, 77, 81]
```

classMarks[0] — список оценок по четырем предметам студента по имени Joe. И нам нужно получить оттуда единственное значение. Как поступить в этом случае? Воспользоваться вторым индексом. Третья из оценок ученика Joe (оценка за чтение) имеет индекс 2. Поэтому, чтобы вывести ее на экран, напишем так:

```
>>> print (classMarks[0][2])  
77
```

Такая запись дает нам первый элемент в списке classMarks (с индексом 0), который представляет собой список оценок ученика по имени Joe, и третий элемент из этого списка (с индексом 2), который представляет собой оценку за чтение. Если в коде вы встречаете имя, за которым следуют два набора квадратных скобок, например classMarks[0][2], скорее всего, вы имеете дело со списком списков.

Приведенная выше таблица на языке математики называется матрицей. Приведенные ниже примеры демонстрируют методы создания матриц и методы работы с ними.

```
#Сумма четных строк и нечетных столбцов  
import random  
n=int(input('Введите размерность квадратной матрицы\n'))  
def create_matrix(n):  
    a=[0]*n # Создаем строку из n нулей  
    for i in range(n):  
        a[i] = [0] * n # Создаем n строк с нулями – получаем матрицу из нулей  
    for i in range(n):  
        for j in range(n):  
            a[i][j]=random.randint(0,99)# Заполняем матрицу случайными числами  
    return(a)  
def my_print(a):  
    b=[0]*n  
    for i in range(n):  
        b[i]= [0]*n  
    for i in range(n):  
        for j in range(n):  
            s=str(a[i][j])  
            if len(s)==1:s=' '+s  
            b[i][j]=s
```

```

    for i in range(n):
        print(b[i])
def sum_chet_nechet(a):
    sch=0
    snch=0
    for i in range(n):
        if i%2==0:
            for j in range(n): sch=sch+a[i][j]
    for j in range(n):
        if j%2==1:
            for i in range(n):
                snch=snch+a[i][j]
    return(sch,snch)
a=create_manrix(n)
my_print(a)
sch,snch=sum_chet_nechet(a)
print('Сумма четных строк = ',sch)
print('Сумма нечетных столбцов = ',snch)
input()

```

Результат:

Введите размерность квадратной матрицы

4

['90', '44', '85', '17']

['20', '53', '69', '49']

['84', '40', '16', '30']

['53', '31', '18', '97']

Сумма четных строк = 406

Сумма нечетных столбцов = 361

Сумма элементов столбца, в котором находится первый минимальный элемент матрицы

```

import random
n=int(input('Введите размерность квадратной матрицы\n'))
def create_manrix(n):
    a=[0]*n
    for i in range(n):
        a[i] = [0] * n
    for i in range(n):
        for j in range(n):
            a[i][j]=random.randint(0,99)
    return(a)
def my_print(a):
    b=[0]*n
    for i in range(n):
        b[i]= [0]*n
    for i in range(n):
        for j in range(n):
            s=str(a[i][j])
            if len(s)==1:s=' '+s
            b[i][j]=s

```

```

    for i in range(n):
        print(b[i])
def my_min(a):
    mm=a[0][0]
    for i in range(n):
        for j in range(n):
            if a[i][j]<mm:
                nj=j
                mm=a[i][j]
    print(nj)
    return(nj)
def sum_min(a,nj):
    s=0
    for i in range(n): s=s+a[i][nj]
    return(s)
a=create_manrix(n)
my_print(a)
nj=my_min(a)
s=sum_min(a,nj)
print('Сумма элементов столбца, в котором минимальный элемент матрицы ',s)
input()

```

Результат:

Введите размерность квадратной матрицы

4

['16', '74', '74', '76']

['93', '48', '55', '97']

['61', ' 4', '48', '57']

['16', '56', '16', ' 4']

1

Сумма элементов столбца, в котором минимальный элемент матрицы 182

### Задания для самостоятельной работы

(решения следует оформлять с помощью функций)

1. Даны два списка. Написать функцию нахождения минимума в списке.
2. Написать функцию, которая находит сумму элементов списка до первого кратного четного элемента.
3. Написать функцию, которая находит сумму элементов списка после последнего четного элемента.
4. Написать функцию нахождения корней в квадратном уравнении.
5. Заполнить с клавиатуры одномерный список и найти сумму его элементов
6. Задан одномерный список случайным. образом. Найти количество чётных элементов на нечетных местах.
7. Задан одномерный список и порядковый номер  $k$ , вводимый с клавиатуры. Подсчитать сколько элементов находится справа или слева.

8. Дана последовательность  $N$  целых чисел:  $a_1, a_2, \dots, a_n$ , получить суммы положительных и отрицательных членов последовательности. Сравнить эти суммы по модулю.
9. Дана последовательность  $N$  целых чисел:  $a_1, a_2, \dots, a_n$ . Найти минимальный и максимальный члены последовательности и указать их номера.
10. Дана последовательность  $N$  целых чисел:  $a_1, a_2, \dots, a_n$ , и задано число  $k$ . Проверить, выполняется ли, что число  $k$  меньше минимального элемента в последовательности.
11. В городе  $N$  банков. Есть информация о стоимости купли-продажи долларов в банках города. Написать программу выбора банков для наиболее выгодного обмена валют.
12. Задано  $N$  судей. Подсчитать среднюю оценку, выставленную этими судьями.
13. Задана матрица. Найти сумму элементов.
14. Задана матрица. Найти сумму строк и указать номер строки, в которой сумма окажется максимальной.
15. Сортировка прямым способом
16. Найти в матрице строку с максимальным элементом и упорядочить эту строку по возрастанию.
17. В матрице чисел размером  $m \times n$  поменять местами  $i$  и  $j$  столбцы.

### 1.10.1 Словари

Вы только что познакомились с таким средством хранения наборов данных в Python, как список. Но в программировании часто возникает необходимость собрать элементы в группу таким образом, чтобы каждому значению было сопоставлено другое значение. Аналогично тому, как телефонная книга связывает имена с номерами телефонов, словарь связывает слово с его определением. В Python *словарь* (dictionary) обеспечивает связывание двух элементов друг с другом. Элементы в этом случае называются *ключом* (key) и *значением* (value). Каждый элемент, или запись в словаре, имеет ключ и значение. Возможно, вы уже слышали про *пары ключ – значение*. Словарь представляет собой коллекцию таких пар.

В качестве простого примера можно рассмотреть список телефонных номеров. Предположим, вам нужно сохранить телефоны друзей. Поиск телефонов будет осуществляться по именам. (Вам повезло, и друзей с одинаковыми именами у вас просто нет.) Имя в этом случае будет *ключом* (элементом, по которому будет выполняться поиск информации), а номер телефона – *значением* (элементом, который вы ищете).

Рассмотрим один из способов создания такого словаря на языке Python. Для начала создадим пустой словарь:

```
>>> phoneNumbers = {}
```

Точно так же мы создавали список, только в данном случае в выражении задействованы фигурные, а не квадратные скобки.

Добавим первую запись:

```
>>> phoneNumbers["Ivan"] = "555-1234"
```

Вот что получится, если теперь вывести содержимое нашего словаря на экран:

```
>>> print (phoneNumbers)
{'Ivan': '555-1234'}
```

Сначала выводится ключ, потом после двоеточия следует значение. Наличие кавычек в данном случае обусловлено тем, что как ключ, так и значение являются строками (но они могут принадлежать и к другому типу данных). То же самое можно сделать и короче:

```
>>> phoneNumbers = {"Ivan": "555-1234"}
```

Внесем в список дополнительные имена. В случае списков мы пользовались методом `append()`, но словари лишены этого удобного инструмента. Поэтому нам остается в явном виде указывать новые ключи и соответствующие им значения:

```
>>> phoneNumbers["Masha"] = "555-6789"
>>> phoneNumbers["Semen"] = "444-4321"
>>> phoneNumbers["Olga"] = "867-5309"
```

Посмотрим на содержимое нашего словаря:

```
>>> print (phoneNumbers)
{'Semen': '444-4321', 'Ivan': '555-1234', 'Masha': '555-6789', 'Olga': '867-5309'}
```

А теперь то, зачем мы, собственно, создавали словарь – поиск элементов. Попробуем найти что-нибудь по имени:

```
>>> print (phoneNumbers["Masha"])
'555-6789'
```

Обратите внимание, что ключ, значение которого мы хотим найти, заключен в квадратные скобки, в то время как для словаря используются фигурные.

Словарь во многом напоминает список. Оба типа являются *коллекциями*, то есть они дают возможность собирать вместе элементы разных типов.

Вот чем они похожи.

- Как списки, так и словари могут содержать элементы любого типа (даже другие списки и словари), что дает вам возможность составлять коллекции чисел, строк, объектов и даже других коллекций.
- Как для списков, так и для словарей существуют методы поиска отдельных элементов.

А вот чем они различаются.

- Списки являются *упорядоченными*. Порядок, в котором вы помещаете элементы в список, сохраняется. Но при этом списки можно подвергать сортировке. Словари же *не упорядочены*. Выведя на экран добавленные в словарь элементы, вы можете обнаружить, что они выводятся вовсе не в том порядке, в котором вы их туда добавляли.
- Доступ к элементам списков осуществляется по индексам, а доступ к элементам словарей – по ключам:

```
>>> print (myList[3])
'eggs'
```

```
>>> print (myDictionary["Джон"])
'555-1234'
```

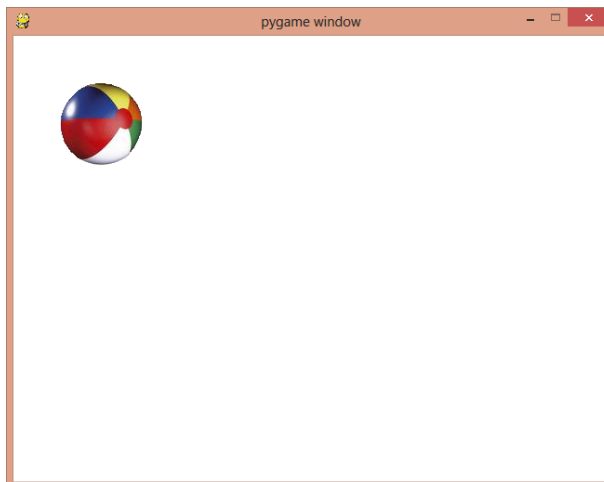
Более подробное изучение свойств словарей находится за пределами данного курса. Но по крайней мере вы получили общее представление о данном типе коллекций.

### 1.11 Сложные рисунки

Рисование фигур, линий и отдельных пикселей является лишь одним из вариантов создания графики. Но иногда возникает необходимость воспользоваться картинкой, взятой в другом месте. Это может быть цифровое фото, графический фрагмент, загруженный из Интернета или созданный вами в графическом редакторе. Простейшим способом работы с такими картинками в модуле Pygame являются функции `image`. Рассмотрим пример:

```
import pygame, sys
pygame.init()
screen = pygame.display.set_mode([640,480])
screen.fill([255, 255, 255])
my_ball = pygame.image.load("beach_ball.png")    # загрузка рисунка из файла
screen.blit(my_ball, [50, 50])                  # вывод на экран
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```

Функция `pygame.image.load()` загружает изображение с диска и создает объект `my_ball`. Этот объект представляет собой поверхность. Но нам она пока не видна, так как существует только в памяти. Единственной видимой нам поверхностью является поверхность отображения `screen`. (Ее мы создали в строке 3.) Строка 6 копирует поверхность `my_ball` на поверхность `screen`. Затем функция `display.flip()` уже знакомым нам способом делает ее видимой.



## 1.12. Списки в компьютерной игре

Как правило, в компьютерной игре участвуют не два (управляемый и неуправляемый), а несколько объектов. Рассмотрим, например, такую ситуацию:



Сюжет этой игры прост: на город (три типа домов) идет кислотный дождь. Спасти город можно, собирая капли дождя в чашку. Одновременно падают пять капель. Для решения этой задачи следует использовать списки. Код программы с необходимыми пояснениями приведен ниже.

```
#Неуправляемое и управляемое движение рисунков (много врагов)
import pygame, sys, random
from pygame import font
pygame.init()
from pygame.color import THECOLORS
font = pygame.font.Font(None, 24)
screenX=1200
screenY=800
screen = pygame.display.set_mode([screenX,screenY])
x=[] # x координаты капель
y=[] # y координаты капель
for i in range (5): # расстановка капель
    x.append(random.randint(10, 1100))
    y.append(100)
dx=0 #капли двигаются по вертикали
dy=[] #у капель разная скогрость
for i in range(5):
    dy.append(random.randint(1,4))
r=75
xr=600
yr=650
width=150
heigth=53
step=20
```



```

n=0
vzriv_file="luja.jpg" #рисунки
drug_file="chasha.jpg"
vrag_file="kapla.jpg"
dom_file=["dom1.jpg","dom2.png","dom3.jpg"]#список рисунков домов
doma=[]#список домов
dom_zel=[]#список признаков, что дом цел
for i in range(3):
    dom=pygame.image.load(dom_file[i])
    doma.append(dom)
for i in range(0,12):
    dom_zel.append(True)
drug=pygame.image.load(drug_file)
vrag=pygame.image.load(vrag_file)
vzriv=pygame.image.load(vzriv_file)
def schet(n):
    text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    return(text)
def move_vrag(x,y,dx,dy,n,r,text):
    for i in range (5):#движение капель с помощью цикла
        x[i]=x[i]+dx
        y[i]=y[i]+dy[i]
        screen.blit(vrag, [x[i],y[i]])
        if (y[i]>=screenY-r):#Если капля попала в дом
            k=x[i]//100
            dom_zel[k]=False #Дом разрушен
            y[i]=50 #новые координаты капли
            x[i]=random.randint(10, 1100)
            n=n-2
            text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
    return(x,y,dx,dy,n,text,dom_zel)
def proverka(x,y,xr,yr,width,height,n,text):
    for i in range(5):#проверка столкновений капли с чашкой
        if (x[i]>=(xr-50))and (x[i]<=(xr+width))and (y[i]>=(yr-r)):#and (y<(yr+height)):
            y[i]=50
            x[i]=random.randint(10, 1100)
            n=n+1
            text=schet(n)
    return(x,y,n,text)
def move_drug(xr,yr,step): #движение чашки
    if event.key == pygame.K_LEFT:
        xr=xr-step
    elif event.key == pygame.K_RIGHT:
        xr=xr+step
    elif event.key==pygame.K_UP:
        yr=yr-step
    elif event.key==pygame.K_DOWN:
        yr=yr+step
    return(xr,yr)
text = font.render(" Очки "+str(n),True, THECOLORS ["blue"])
delay = 100
interval = 50

```

```

pygame.key.set_repeat(delay, interval)
running = True
while running:
    screen.fill(THECOLORS['white'])
    screen.blit(text, [250, 50] )
    screen.blit(drug, [xr,yr])
    k=0
    for i in range(0,12): #если дом цел, рисуем дом
        if dom_zel[i]:
            screen.blit(doma[k], [10+i*100,730])
            k=k+1
            if k==3:k=0
        else: screen.blit(vzriv, [10+i*100,730]) #если разрушен, рисуем взрыв и пожар
    x,y,dx,dy,n,text,dom_zel=move_vrag(x,y,dx,dy,n,r,text)
    x,y,n,text=proverka(x,y,xr,yr,width,height,n,text)
    pygame.time.delay(70)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            xr,yr=move_drug(xr,yr,step)
    pygame.display.flip()
pygame.quit()

```

Списки в том или ином виде используются в абсолютном большинстве компьютерных игр

## 1.12. Методы и средства работы с текстовой информацией. Строки

Строки в Python - упорядоченные последовательности символов, используемые для хранения и представления текстовой информации, поэтому с помощью строк можно работать со всем, что может быть представлено в текстовой форме. Мы уже пользовались строками. В Python любой текст, заключенный в апострофы представляет собой строку.

Над строками определены следующие базовые операции:

Конкатенация (сложение)

```

>>> S1 = 'spam'
>>> S2 = 'eggs'
>>> print(S1 + S2)

```

Результат:

```
spameggs
```

Дублирование строки

```

>>> print('spam' * 3)
spamspamspam

```

Функция len() возвращает количество символов (длину) в строке:

```
>>> len('spam')
4
```

Доступ к символам по индексу:

```
>>> S = 'spam'
>>> S[0]
's'
>>> S[2]
'a'
>>> S[-2]
'a'
```

Как видно из примера, в Python возможен и доступ по отрицательному индексу, при этом отсчет идет от конца строки.

Извлечение среза

Оператор извлечения среза: [X:Y]. X – начало среза, а Y – окончание; символ с номером Y в срез не входит. По умолчанию первый индекс равен 0, а второй - длине строки.

```
>>> s = 'spameggs'
>>> s[3:5]
'me'
>>> s[2:-2]
'ameg'
>>> s[:6]
'spameg'
>>> s[1:]
'pameggs'
>>> s[:]
'spameggs'
```

Кроме того, можно задать шаг, с которым нужно извлекать срез.

```
>>> s[::-1]
'sggemaps'
>>> s[3:5:-1]
''
>>> s[2::2]
'aeg'
```

Проверка на вхождение подстроки в строку

```
S.find(str) Поиск подстроки в строке. Возвращает номер первого вхождения или -1
s='5'
s1='456'
n=s1.find(s)
>>> print(n)
1
```

**S.rfind(s)** Поиск подстроки в строке. Возвращает номер последнего вхождения или -1

```
s='5'
s1='4567859'
>>> n=s1.rfind(s)
>>> print(n)
5
```

**S.index(s)** Поиск подстроки в строке. Возвращает номер первого вхождения или вызывает ValueError

**S.rindex(s)** Поиск подстроки в строке. Возвращает номер последнего вхождения или вызывает ValueError

**S.split(символ)** Разбиение строки по разделителю

```
s='4 5'
>>> s1=s.split(' ')#Разбиение по пробелу
>>> print(s1)
['4', '5']
s1=s.split()# Разбиение по пробелу, символу конца строки и т.д.
>>> print(s1)
['4', '5']
```

**chr(число)** Преобразует код ASCII в символ

```
ss=chr(97)
>>> print(ss)
a
```

**ord(символ)** Преобразует символ в его код ASCII

```
>>> ord('a')
97
>>>
```

Пример 1.

```
#В строке заменить точки и запятые на !
def znak_probел(s):
    s1=""
    for i in range(len(s)):
        if (s[i]==',' or s[i]=='.') :
            s1=s1+'!'
        else: s1=s1+s[i]
    return(s1)
s='Из двух строк, А и В по n слов в каждой, сформировать строку С, записать в нее
.поочередно слова из строк А и В.'
s=znak_probел(s)
print(s)
input()
```

Результат:

Из двух строк! А и В по n слов в каждой! сформировать строку С! записать в нее поочередно слова из строк А и В!

Пример 2.

#В заданном предложении подсчитать количество и выдать слова, содержащие 2 буквы «а»

```
def bukvi_a(s):
    slova=s.split(' ')
    n=0
    otvet=[]
    for i in range(len(slova)):
        s1=slova[i]
        k=0
        for j in range(len(s1)):
            if s1[j]=='a': k=k+1
        if k>=2:
            n=n+1
            otvet.append(s1)
    return(n,otvet)
s='Маша ела кашу. Мама мыла рама'
#print(len(s))
#print(s[len(s)-1])
n,slova=bukvi_a(s)
print(n)
print(slova)
input()
```

Результат:

```
3
['Маша', 'Мама', 'рама']
```

### 1.13 Текстовые файлы

Рассмотрим способы организации долговременного хранения информации и основные операции для работы с текстовыми файлами.

Начнем с чтения информации из текстового файла.

С этой целью создадим в Блокноте Windows, файл students.txt, который будем вычитывать. Вот его содержимое:

Каплин Матвей  
Рубан Василина  
Бирюкова Аврора  
Плахотин Егор

Прежде чем начать читать/писать файл необходимо его открыть. С этого начинается любая программа для работы с файлами.

Например:

```
students=open('students.txt','r')
```

При этом создается так называемая файловая переменная или, иначе говоря, файловый объект (в данном случае students). Все дальнейшие операции с файлом выполняются с ее помощью.

- Режимы доступа к текстовому файлу
- “r” Чтение из текстового файла. Если файла не существует, Python сообщит об ошибке.
- “w” Запись в текстовый файл. Если файл существует, его содержимое будет заменено. Если файл не существует, он будет создан.
- “a” Дозапись в текстовый файл. Если файл существует, новые данные будут дописаны в конец. Если файл не существует, он будет создан.

Посимвольное чтение из текстового файла:

```
students=open('students.txt','r')
lines=students.read(1)
students.close() #Файл нужно закрывать
print(lines)
input()
```

Результат:

К

```
#Читаем 6 символов:
students=open('students.txt','r')
lines=students.read(6)
students.close()
print(lines)
input()
```

Результат:

Каплин

```
#Читаем 6 символов и затем еще 7 символов, затем возвращаемся в начало:
students=open('students.txt','r')
lines=students.read(6)
print(lines)
lines=students.read(7)
print(lines)
students.seek(0) #Возвращение к символу номер 0. или к любому другому символу
lines=students.read(6)
print(lines)
students.close()
input()
```

Результат:

Каплин

Матвей

Каплин

```
#Читаем весь файл сразу:
students=open('students.txt','r')
lines=students.read()
print(lines)
students.close()
input()
```

Результат:  
Каплин Матвей  
Рубан Василина  
Бирюкова Аврора  
Плахотин Егор

```
#Чтение строки:
students=open('students.txt','r')
lines=students.readline()
print(lines)
students.close()
input()
```

Результат:  
Каплин Матвей

Использование метода **readlines()**, читающего текстовый файл в список, элементами которого являются строки:

```
students=open('students.txt','r')
lines=students.readlines()
print(lines)
students.close()
input()
```

Результат:  
['Каплин Матвей\n', 'Рубан Василина\n', 'Бирюкова Аврора\n', 'Плахотин Егор']

Запись информации в текстовый файл.

Сначала необходимо открыть файл в требуемом режиме: `text_file = open("write_it.txt", "w")`. Файл `write_it.txt` создается как пустой текстовый файл, готовый к записи какого-либо текста из программы. Если бы файл `write_it.txt` уже существовал, то на замену ему был бы создан пустой файл, а все предыдущее содержимое утратилось бы. Вслед за тем применяем метод файлового объекта **write()**, который записывает строку в файл:

```
text_file.write("Строка 1\n")
text_file.write("Это строка 2\n")
text_file.write("Этой строке достался номер 3\n")
```

Метод **write()** не вставляет символ перехода на новую строку автоматически после каждой строки, которую он пишет в файл. Символы `\n` вы можете помещать там, где хотите

видеть их в файле. Например, если убрать их из предшествующего кода, то программа записала бы одну длинную строку текста.

Не обязательно заканчивать строку, которая пишется в файл, символом `\n`. Конечный результат остался бы прежним, если бы сформировали единую строку *"Строка 1 \ n Это строка 2\n Этой строке достался номер 3\n"* и записана она в файл одним-единственным вызовом метода **write()**.

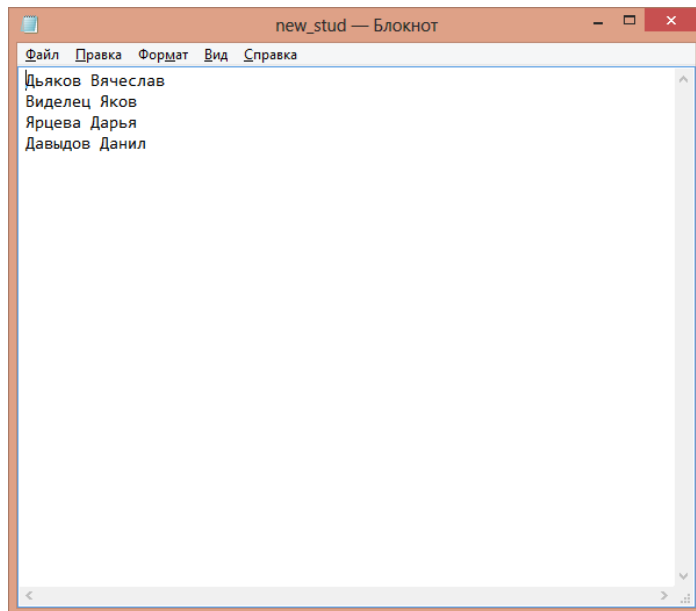
После этого закрываем файл:

```
text_file.close()
```

*#Создадим новый файл с фамилиями студентов:*

```
students=open('new_stud.txt','w')
students.write('Дьяков Вячеслав\n')
students.write('Виделец Яков\n')
students.write('Ярцева Дарья\n')
students.write('Давыдов Данил\n')
students.close()
```

Результат:



*#Запись списка строк в файл:*

```
students=open('new_stud.txt','w')
lines=['Дьяков Вячеслав\n','Виделец Яков\n','Ярцева Дарья\n','Давыдов Данил\n']
students.writelines(lines)
students.close()
```

Результат прежний

Пример:

*#Найти файле students.txt самую длинную фамилию*

```
students=open('students.txt','r')
lines=students.readlines()
print(lines)
```



```

students.close()
smax=[]
for i in range(len(lines)):
    s=lines[i].split(' ')
    s=s[0]
    if len(s)>len(smax):smax=s
print(s)
input()

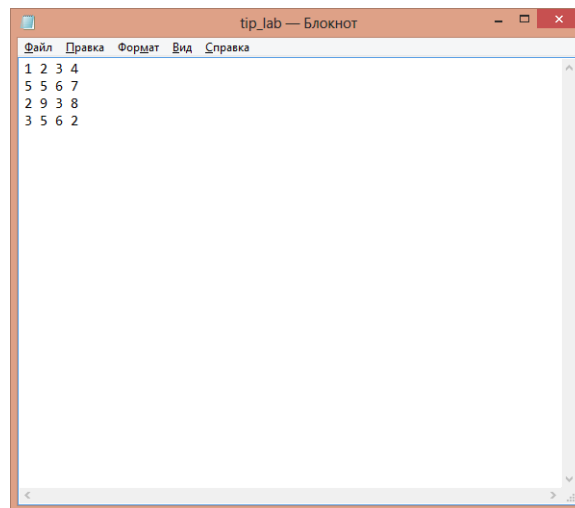
```

Результат:

['Каплин Матвей\n', 'Рубан Василина\n', 'Бирюкова Аврора\n', 'Плахотин Егор']  
Плахотин

Следующий пример показывает способ чтения матрицы из текстового файла и преобразование в матрицу целых чисел (заготовка для игры «Лабиринт»).

Исходный файл. Обратите внимание на пробелы 1 2 3:



```

#Заготовка для лабиринта
sp=[0]*4 #Формируем строку нулей
for i in range(4):sp[i]=[0]*4 #Размножаем строки вниз
lab=open('tip_lab.txt','r')
for i in range(4):
    lines=lab.readline() #Читаем файл по строкам
    s=lines.split() #Удаляем из строки пробелы и символы конца строки
    print(s)
    for j in range(4):
        sp[i][j]=int(s[j]) #Записываем строку s в i-ю строку списка sp
lab.close()
for i in range(4):print(sp[i]) #Печатаем матрицу sp
input()

```

Матрица после обработки split()

```
['1', '2', '3', '4']  
['5', '5', '6', '7']  
['2', '9', '3', '8']  
['3', '5', '6', '2']
```

Итоговая матрица готова. В ней закодирован лабиринт:

```
[1, 2, 3, 4]  
[5, 5, 6, 7]  
[2, 9, 3, 8]  
[3, 5, 6, 2]
```

#### 1.14. Хранение структурированных данных в файлах. Модуль pickle

В разделе 1.13 речь шла об операциях чтения и записи с текстовыми файлами. Но это лишь один из способов хранения данных на жестком диске. А что делать, если требуется сохранить список или объект?

Список иногда может состоять из строк, но так бывает далеко не всегда. А как быть с сохранением объектов? Нужно преобразовать все свойства объекта в строки и записать их в текстовый файл. Но тогда вам в дальнейшем потребуется совершить обратную операцию по восстановлению объекта, что может оказаться непростой задачей.

К счастью, в Python существует более простой способ сохранения списков и объектов. Для этого применяется модуль `pickle`. Это забавное название переводится как «маринованный огурец». Но давайте задумаемся. Маринование — это способ сохранения пищи для использования в будущем. В Python вы «маринуете» свои данные, чтобы получить возможность сохранить их на диске и использовать в дальнейшем. Все вполне логично!

Представьте, что у вас есть список различных элементов:

```
my_list = ['Leonid', 73, 'Hello', 81.9876e-13]
```

Для применения команды `pickle` первым делом следует импортировать одноименный модуль:

```
import pickle
```

Затем, чтобы «замариновать» что-нибудь, например, список, используйте функцию `dump()`. Это английское слово означает «сброс». Имя функции легко запомнить, если представить, как огурцы кладут в банку. Функция `dump()` работает с файловыми объектами, и вы уже знаете, каким способом они создаются:

```
pickle_file = open('my_pickled_list.pkl', 'wb')#wb (байт)используется для записи в двоичный файл  
#Для чтения – rb
```

Теперь остается преобразовать объект в поток байтов (эта процедура называется *сериализацией*) и сохранить их в файле:

```
pickle.dump(my_list, pickle_file)
```

Целиком процесс выглядит так:

```
import pickle
my_list = ['Leonid', 73, 'Hello', 81.9876e-13]
pickle_file = open('my_pickled_list.pkl', 'w')
pickle.dump(my_list, pickle_file)
pickle_file.close()
```

Метод подходит для сохранения в файле всех структур данных. Но как извлечь их назад?

В реальной жизни маринованные овощи невозможно превратить в свежие. Маринование – операция необратимая. Но в Python вы можете не только «сохранить» свои данные с помощью сериализации, но и выполнить обратную операцию – *десериализацию* – и вернуть данным первоначальный вид.

Десериализация выполняется с помощью функции `load()`. Ей передается файловый объект с сериализованными данными, а она возвращает их в исходное состояние. Попробуем это на практике. После запуска предыдущей программы в папке с этой программой должен был появиться файл `my_pickled_list.pkl`. Запустив следующую программу вы получите назад свой список.

```
import pickle
pickle_file = open('my_pickled_list.pkl', 'rb')
recovered_list = pickle.load(pickle_file)
pickle_file.close()
print recovered_list
```

Вы должны получить такой результат:

```
['Leonid', 73, 'Hello', 8.1987599999999997e-012]
```

Таким путем вы сможете сохранять списки и получать их обратно для дальнейшей работы с ними.

## Глава 2. Виды игровых программ и примеры их реализации

### 2.1 Заставка, помощь меню

У любой компьютерной игры существует «обрамление» в виде заставки, помощи и меню, управляющего всей игрой. В этом параграфе мы разберем принципы создания этих фрагментов игровой программы.

Начнем с заставки. Она у нас будет оформлена в виде отдельной модуля, работа которого будет предварять основную игровую программу. Вообще говоря, содержание заставки полностью зависит от фантазии разработчика. Мы остановимся на простейшем ее варианте. В заставке будет выводиться на экран название игры и информация о ее разработчике. Например:



Возможный код модуля *zastavka*:

```
import pygame, sys
from pygame.color import THECOLORS
from pygame import font
pygame.init()
pygame.display.set_caption("Крутая игра")
def zastavka1():
    font = pygame.font.Font(None, 45)
    text = font.render("Welcome to Cosmos", True, THECOLORS ["blue"])
    screen = pygame.display.set_mode([800, 600])
    background = pygame.Surface(screen.get_size())
    background.fill([255,255,255])
    cosm_file="cosm.jpg"
    location = [580, 350]
    earth_file="earth.jpg"
    earth=pygame.image.load(earth_file)
    cosm=pygame.image.load(cosm_file)
    raceta_file="rac_zast.jpg"
    raceta=pygame.image.load(raceta_file)
    screen.blit(background, (0, 0))
    screen.blit(cosm, location)
    screen.blit(text, [250, 50])
    font = pygame.font.Font(None, 25)
    text = font.render("Press any key", True, THECOLORS ["black"])
    screen.blit(text, [350, 550])
    screen.blit(raceta, [100, 300])
    screen.blit(earth, [300, 100])
    pygame.display.flip()
    running=True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                running = False
```

Текст помощи следует набрать в Блокноте и сохранить как текстовый файл. Содержанием должны быть сюжет и правила игры. Примерный код модуля *my\_help* приведен ниже.

```

import pygame, sys
from pygame.color import THECOLORS
from pygame import font
pygame.init()
def myhelp():
    pygame.display.set_caption("Крытая изра")
    screen = pygame.display.set_mode([600, 600])
    background = pygame.Surface(screen.get_size())
    background.fill([0,0,0])
    color = THECOLORS["white"]
    top=10; left=20
    font = pygame.font.Font(None, 20)
    my_file = open('myhelp.txt', 'r')#Открываем файл для чтения
    lines = my_file.readlines()#Записываем строки из файла в список lines
    my_file.close()
    dlina=len(lines)#Это количество строк
    for i in range(0,dlina):
        ln=lines[i]
        dl=len(ln)-1 #Удаляем символ конца строки
        ln=ln[0:dl]
        text = font.render(ln,1, color)
        background.blit(text, [left, top] )
        top=top+20
    text = font.render("Press any key",True, color)
    background.blit(text, [150, 550])
    screen.blit(background, (0, 0))
    pygame.display.flip()
    running=True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                running = False
    screen = pygame.display.set_mode([400, 600])

```

Модуль меню управляет всей игрой. Сама игра должна храниться в отдельном модуле с именем, например, game:



Код модуля my\_menu:

```
import pygame, sys, random, zastavka
import pickle
from pygame.color import THECOLORS
from pygame import font
pygame.init()
zastavka.zastavka1()
pygame.display.set_caption("Крутая игра")
screen = pygame.display.set_mode([400, 600])
background = pygame.Surface(screen.get_size())
top=[60,210,360,510]; left=100; width=200; height=80
menu_text=["Play", "Help", "Exit", "Records"]
color = THECOLORS["blue"]
font = pygame.font.Font(None, 40)
line_width = 0
running=True
while running:
    background.fill([255,200,0])
    for i in range(0,4):#рисует пункты меню
        pygame.draw.rect(background, color, [left, top[i], width, height], line_width)
        text = font.render(menu_text[i],1, THECOLORS ["white"])
        background.blit(text, [left+50, top[i]+30] )
    text = font.render("Menu",1, THECOLORS ["blue"])
    background.blit(text, [150,10] )
    screen.blit(background, (0, 0))
    pygame.display.flip()
    for event in pygame.event.get():
        if event.type == pygame.MOUSEBUTTONDOWN:
            (x,y)=pygame.mouse.get_pos()#координаты курсора мыши
            for i in range (0,4):
                if x>left and x<left+width and y>top[i] and y<top[i]+height:
                    break
            if i==2:
                running = False #выход
```

```

if i==0:
    import game
    game.mygame() #игра
if i==1:
    import my_help
    my_help.myhelp() #помощь

if i==3:#вывод на экран таблицы рекордов
    pickle_file = open('my_rec.pkl', 'rb')
    names = pickle.load(pickle_file)
    pickle_file.close()
    import rec_show
    rec_show.show_rec(names)
pygame.quit()

```

## 2.2 Таблица рекордов

Для создания и таблицы рекордов и работы с ней удобно использовать модуль pickle.

Сначала создаем список с именами игроков и количеством их очков. Например, такой:

```
names=["Unknown","Unknown","Unknown","Unknown","Unknown",0,0,0,0,0,]
```

С помощью короткой программы создаем файл и одновременно заносим в него список:

```

import pickle
names=["Unknown","Unknown","Unknown","Unknown","Unknown",0,0,0,0,0,]
pickle_file = open('my_rec.pkl', 'wb')#открытие файла для записи
pickle.dump(names, pickle_file)
pickle_file.close()

```

Все! Файл существует. В дальнейшем в нем появятся реальные имена и очки. Теперь его нужно вывести на экран.

Этот модуль – демонстрация таблицы рекордов:

```

import pickle
import pygame, sys
from pygame.color import THECOLORS
from pygame import font
pygame.init()
pickle_file = open('my_rec.pkl', 'rb')
names = pickle.load(pickle_file)#список содеожит имена и очки
pickle_file.close()
def show_rec(names):
    nam=names[0:5]
    scores=names[5:]
    pygame.display.set_caption("Крутая игра")
    screen = pygame.display.set_mode([400, 600])
    background = pygame.Surface(screen.get_size())
    background.fill([255,255,255])

```

```

color = THECOLORS["red"]
left=300
top=10
font = pygame.font.Font(None, 32)
dlina=len(nam)
for i in range(0,dlina):
    sc=str(scores[i])
    text = font.render(sc,1, color)
    background.blit(text, [left, top] )
    top=top+40
top=10; left=20
for i in range(0,dlina):
    nm=nam[i]
    text = font.render(nm,1, color)
    background.blit(text, [left, top] )
    top=top+40
text = font.render("Press any key",True, color)
background.blit(text, [150, 550])
screen.blit(background, (0, 0))
pygame.display.flip()
running=True
while running:
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            running = False

```

Предположим, что по окончании игры известно имя игрока (my\_name) и набранные им очки. Ввод имени (информации в графическом режиме) мы рассмотрим ниже. Следующая код решает две задачи: определяет вошел ли игрок в таблицу рекордов и сортирует таблицу с учетом результата игрока. Его следует разместить в модуле game после основного цикла while.

```

myn=my_name
och=n
pickle_file = open('my_rec.pkl', 'rb')
names = pickle.load(pickle_file)
pickle_file.close()
nam=names[0:5]
scores=names[5:]
scores.append(och)#добавляем в список очки игрока
scores.sort(reverse=True)#Сортируем список по убыванию
del scores[len(scores)-1]#Удаляем последний элемент списка
if och in scores:#если очки остаются в списке
    k=scores.index(och)#находим порядковый номер очков
    nam.insert(k,myn)#Вставляем имя игрока на позицию очков
    del nam[len(scores)-1]#удаляем последнее имя
    nam.extend(scores)#объединяем имена и очки в один список
    names=nam
    pickle_file = open('my_rec.pkl', 'wb')#записываем новую таблицу в файл
    pickle.dump(names, pickle_file)
    pickle_file.close()
import rec_show

```



```

rec_show.show_rec(names)#Выводим на экран новую таблицу
else:#Если не попали в рекорды
    background.fill([255,255,255])
    font = pygame.font.Font(None, 50)
    text = font.render("Не вошли в рекорды",True, THECOLORS ["red"])
    screen.blit(background,(0, 0))
    screen.blit(text, [10, 250] )
    pygame.display.flip()
    pygame.time.delay(4000)

```

## 2.3 Ввод информации в графическом режиме

Во многих играх необходимо в процессе игры вводить имя игрока, уровень сложности игры и так далее. Рассмотрим эти вопросы. Начнем с ввода имени. Ниже приведен примерный код с комментариями.

```

#Этот файл предлагает игроку ввести имя:
import pygame, sys
from pygame.color import THECOLORS
from pygame import font
pygame.init()
pygame.display.set_caption("Крутая игра")
name=""
def direction(name):
    screen = pygame.display.set_mode([300, 200])
    background = pygame.Surface([300,160])
    info_w = pygame.Surface([300,40])
    info_w.fill([255,255,255])
    color = THECOLORS["blue"]
    font = pygame.font.SysFont("Arial", 24,False,False)
    color = THECOLORS["blue"]
    myfont = pygame.font.SysFont("Arial", 16,False,False)
    info_text=font.render("Введите ваше имя",1, THECOLORS ["red"])
    info_w.blit(info_text,[50,10])
    screen.blit(info_w, (0, 0))
    text = font.render("",1, THECOLORS ["blue"])
    my_text = myfont.render("Press Enter",1, THECOLORS ["black"])
    k=0#Эта переменная нужна для ввода заглавных букв. Shift здесь не работает
    hor=80
    running=True
    while running:
        background.fill([255,200,0])
        background.blit(my_text, [hor+20,120] )
        background.blit(text, [hor,50] )
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key==pygame.K_ESCAPE:
                    running = False
            else:
                st=pygame.key.name(event.key)
                if k==0 and not (st=="return"):#Фамилия начинается с заглавной буквы
                    st=st.title()

```

```

        k=1
    if st=="space":
        st=" "
        k=0#После пробела имя начинается с заглавной буквы
    if st=="backspace":
        st=""
        a=len(s)-1
        s=s[0:a]
    if not(st=="return"):#Если не нажата Enter
        s=s+st
    if st=="return":#Если нажата Enter
        name=s
        running=False
        text = font.render(s,0, THECOLORS ["blue"])
    screen.blit(background, (0, 40))
    pygame.display.flip()
    return(name)

```

Программа ввода чисел принципиально не отличается от вышеприведенной. В данном случае нет необходимости заботиться о заглавных буквах. После ввода следует преобразовать строковую переменную в целую или вещественную.

## 2.4 Стрельба в компьютерной игре

Во многих динамических играх используется стрельба. Пуля, выпущенная игроком, должна поразить «противника». Обычно выстрел производится при нажатии клавиши «пробел». Далее: при нажатии этой клавиши изображение пули должно появиться вблизи орудия. Само движение пули – это неуправляемое движение, которое начинается в момент, когда мы нажали пробел.

Нужно каким-то образом отметить, что выстрел произведен, и пуля может лететь. С этой целью уместно использовать переменную типа *Boolean* с именем, например, *vistrel*

```
Vistrel=False #Выстрел не произведен
```

Реакцию на нажатие пробела уместно включить в процедуру управляемого движения `move_drug()`.

```

def move_drug(xr,yr,xv,yv,vistrel,step):
    if event.key == pygame.K_LEFT:
        xr=xr-step
    elif event.key == pygame.K_RIGHT:
        xr=xr+step
    elif event.key==pygame.K_SPACE:
        if (vistrel==False):
            vistrel=True
            xv=xr+80
            yv=yr-20
            screen.blit(racet, [xv,yv])
    return(xr,xv,yv,vistrel)

```

Процедура, реализующая полет пули достаточно проста:

```
def polet(x,y,xv,yv,vistrel,vz):
    for i in range(5):
        yv[i]=yv[i]-5
        screen.blit(racet, [xv[i],yv[i]])
        if yv[i] <=50: vistrel[i]=False
        If (xv[i]>=x) and (xv[i]<=x+50) and (yv[i]<=y+20):
            vistrel[i]=False
            vz=True #Эта переменная отвечает за взрыв при попадании во врага
            y=50
            x=random.randint(10, 1100)
    return(x,y,yv,vistrel,vz)
```

Если у игрока в наличии имеется много пуль или снарядов, необходимо создавать соответствующие списки.

## 2.5. Лабиринт

Популярной компьютерной игрой является лабиринт. Лабиринт должен быть заполнен различными сюрпризами. Игрок, перемещаясь по лабиринту, попадает в различные ситуации, из которых ему предстоит выбраться. Содержательная часть игры целиком зависит от фантазии разработчика. Фантазируйте в свое удовольствие! Мы же здесь на простейшем примере, продемонстрируем технологию разработки такого рода игр. Пример состоит в следующем: по лабиринту двигается персонаж. Если он наткнется на стенку, у него отнимаются очки. Если он находит приз, то очки прибавляются. Задача игры – выйти из лабиринта и набрать заданное количество очков.

Разработка игры начинается с проектирования лабиринта. Мы просто рисуем его в тетради в клетку. Далее, помечаем различные элементы лабиринта цифрами. (В нашем случае - 0 – стена, 1 – дорога, 2 – герой, 3 - приз, 4 - выход. Затем создаем в нашей программе двумерный список, отображая, таким образом, наш лабиринт. Например, так:

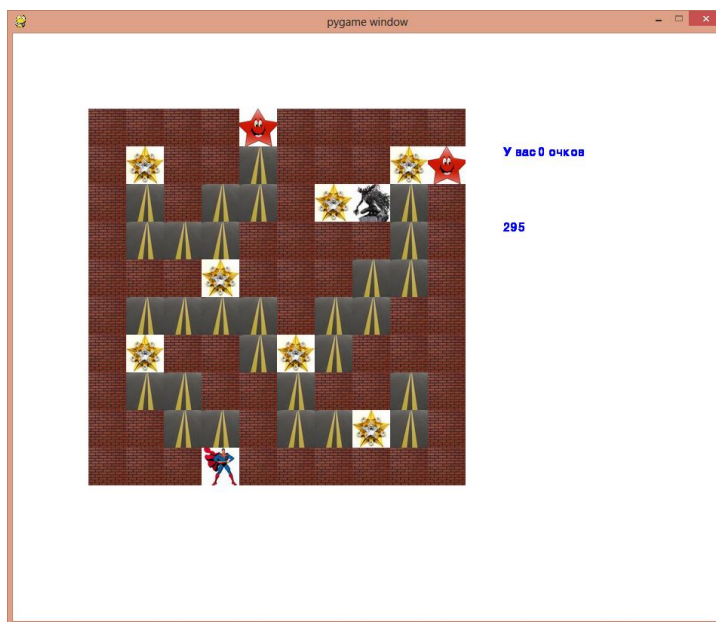
```
import pygame
pygame.init()
pygame.mixer.init()
zvuk=pygame.mixer.Sound("hit_wall.wav")#звук, если уперся в стенку
from pygame.color import THECOLORS
from pygame import font
pygame.time.set_timer(pygame.USEREVENT, 1000) #Это таймер
font = pygame.font.Font(None, 24)
lab1=[[0,0,0,0,4,0,0,0,0,0],
       [0,3,0,0,1,0,0,0,3,4],
       [0,1,0,1,1,0,3,1,1,0],
       [0,1,1,1,0,0,0,0,1,0],
       [0,0,0,3,0,0,0,1,1,0],
       [0,1,1,1,0,1,1,0,0],
       [0,3,0,0,1,3,1,0,0,0],
       [0,1,1,0,0,1,0,0,1,0],
       [0,0,1,1,0,1,1,3,1,0],
       [0,0,0,2,0,0,0,0,0,0]]
#Далее загружаем картинки
geroy_file='geroy.jpg'
stena_file='stena.jpg'
```

```

doroga_file='doroga.jpg'
priz_file='priz.jpg'
vihod_file='star.jpg'
geroy=pygame.image.load(geroy_file)
stena=pygame.image.load(stena_file)
doroga=pygame.image.load(doroga_file)
priz=pygame.image.load(priz_file)
vihod=pygame.image.load(vihod_file)
#Формируем экран и инициализируем переменные
screen = pygame.display.set_mode([940,780])
screen.fill([255,255,255])
nx=100 #Расстояния от края графического окна до лабиринта
ny=100
dx=50 #размеры клетки
dy=50
ochki=0
tim=300 #отведенное на игру время
klad=7
#Функция drawlab() рисует лабиринт

```

Лабиринт будет выглядеть следующим образом:



```

def drawlab(ochki):
    for i in range(0,10):
        for j in range(0,10):
            x=nx+dx*j
            y=ny+dy*i
            if lab1[i][j]==0:
                screen.blit(stena,[x,y])
            elif lab1[i][j]==1:
                screen.blit(doroga,[x,y])
            elif lab1[i][j]==2:
                screen.blit(geroy,[x,y])
            ik=i # Запоминаем номер строки и столбца героя

```

```

        jk=j
    elif lab1[i][j]==3:
        screen.blit(priz,[x,y])
    elif lab1[i][j]==4:
        screen.blit(vihod,[x,y])
text=font.render('У вас '+str(ochki)+' очков',1, THECOLORS ["blue"])
screen.blit(text,[650,150])
text=font.render(str(tim),1, THECOLORS ["blue"])
screen.blit(text,[650,250])
pygame.display.flip()
return(ik,jk,font,ochki)
#Следующая функция анализирует клавиши
def analyskey(i,j,ik,jk):
    if event.key == pygame.K_UP:
        i=ik-1
    elif event.key == pygame.K_DOWN:
        i=ik+1
    elif event.key == pygame.K_LEFT:
        j=jk-1
    elif event.key == pygame.K_RIGHT:
        j=jk+1
    return(i,j)
#Функция движения в лабиринте
def movelab(i,j,ik,jk,lab1,ochki):
    x=nx+dx*j
    y=ny+dy*i
    xk=nx+dx*jk
    yk=ny+dy*ik
    if lab1[i][j]==0:
        i=ik
        j=jk
        zvuk.play()
    if lab1[i][j]==1:
        lab1[i][j]=2
        lab1[ik][jk]=1
        ik=i
        jk=j
    if lab1[i][j]==3:
        lab1[i][j]=2
        lab1[ik][jk]=1
        ik=i
        jk=j
        ochki=ochki+1
        screen.fill([255,255,255])
    if lab1[i][j]==4:
        lab1[i][j]=2
        lab1[ik][jk]=1
        ik=i
        jk=j
        if ochki<klad:
            text=font.render("Вы не собрали призы",1, THECOLORS ["blue"])
            screen.blit(text,[450,50])

```

```

    ik,jk,font,ochki= drawlab(ochki)
return(i,j,ik,jk,lab1,ochki)
ik,jk,font,ochki =drawlab(ochki)
i=ik
j=jk
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    if event.type == pygame.KEYDOWN:
        i,j= analyskey(i,j,ik,jk)
        i,j,ik,jk,lab1,ochki= movelab(i,j,ik,jk,lab1,ochki)
    if event.type == pygame.USEREVENT:#Счет времени
        tim=tim-1
        pygame.draw.rect(screen, [255,255,255], [650, 250, 100, 30], 0)
        text=font.render(str(tim),0, THECOLORS ["blue"])
        screen.blit(text,[650,250])
    if tim==0:
        text=font.render("Ваше время истекло",1, THECOLORS ["blue"])
        screen.blit(text,[450,50])
        pygame.display.flip()
        pygame.time.delay(3000)
        running=False
    pygame.display.flip()
pygame.quit()

```

Предложенная программа является просто учебным примером. Разумеется, возможны различные ее усложнения, введение врагов, создание усложняющихся уровней и т.д.

## Заключение

Заканчивая книгу, хочется подчеркнуть, что ваше будущее зависит от вас. Мы не коснулись здесь ряда важнейших разделов программирования, и поэтому вам стоит продолжить свое образование. В жизни существуют не только игровые программы, и вам предстоит познакомиться со многими интересными приложениями программирования. Но если вы в результате вашей работы почувствовали интерес и вкус к профессии программиста, если вы приобрели навыки самостоятельной разработки программ, то мы будем считать свою задачу выполненной. Успехов вам в вашей профессиональной деятельности!

## Литература

1. Л.А. Москвина, Л.К. Попов. Обучение программированию в процессе создания компьютерной игры. Новосибирск, 2005
2. А.Н. Салтовский. Python – азбука программирования. Новосибирск, 2018
3. У. Сэнд, К. Сэнд. Hello World! Занимательное программирование. – СПб.: Питер, 2016. – 400 с.
4. С. Шапошникова. Основы программирования на Python. <http://younglinux.info>, 2011